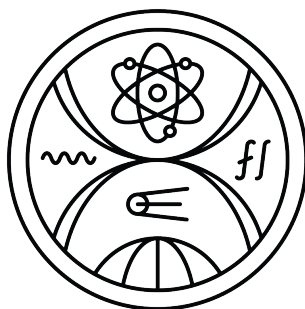


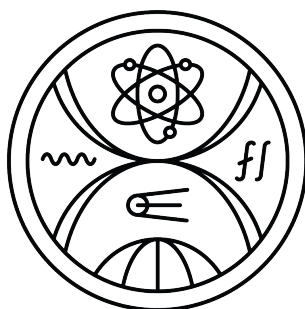
Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky



MODELOVANIE SOFTVÉROVÝCH ŠTÝLOV A VZOROV

Diplomová práca

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky



MODELOVANIE SOFTVÉROVÝCH ŠTÝLOV A VZOROV

Diplomová práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Aplikovaná informatika
Školiace pracovisko::	Katedra aplikovanej informatiky
Školiteľ:	Ing. Lukáš Radoský
Konzultant:	



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Anna Lenhardtová
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Modelovanie softvérových štýlov a vzorov
Modelling software styles and patterns

Anotácia: Vo vývoji softvéru vznikajú zadania, problémy a situácie, ktoré sa naprieč projektami opakujú. Softvérové inžinierstvo sa takýmito javmi zaoberá a ponúka na nie riešenia. Pre problémy celkovej štruktúry softvérových systémov ponúka architektonické štýly, ktoré na vysokej úrovni abstrakcie definujú štruktúru softvéru. Príkladom sú Pipes&Filters, Blackboard, mikroslužby a podobne. Pre často sa opakujúce problémy na nižšej úrovni granularity ponúka návrhové vzory, napríklad Composite, Abstract Factory a podobne.

Analyzujte možnosti modelovania architektonických štýlov a návrhových vzorov na rovnakej úrovni granularity, napríklad na úrovni diagramu tried. Vytvorte príklady takýchto modelov pre vybrané štýly a vzory. Overte vhodnosť vašich modelov, napríklad ich vykonateľnosťou.

Cieľ: Návrh spôsobu modelovania zvoleného architektonického štýlu či štýlov na nízkej úrovni abstrakcie
Vytvorenie príkladov modelov pre zvolenú množinu štýlov a vzorov

Literatúra: JOUAULT, Frédéric, et al. Designing, animating, and verifying partial UML Models. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems. 2020. p. 211-217.

Buschmann F. et al.: Pattern-oriented software architecture: a pattern language for distributed computing, Vol. 4. New York : John Wiley & Sons, 2007.

GAMMA, E.: HELM, R., JOHNSON, R., VLISSIDES, J. (1994), Design Patterns – Elements of reusable object-oriented software code, Addison-Wesley

Richards, M., and Ford, N. 2020. . Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media, Incorporated.

Poznámka: architektonické štýly, návrhové vzory, softvérové inžinierstvo



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Vedúci: Ing. Lukáš Radoský
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 18.11.2024

Dátum schválenia: 21.11.2024

prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Čestne vyhlasujem, že celú diplomovú prácu na tému „Modelovanie softvérových štýlov a vzorov“, vrátane všetkých jej príloh a obrázkov, som vypracovala samostatne, a to s použitím literatúry uvedenej v priloženom zozname a nástrojov umelej inteligencie. Vyhlasujem, že nástroje umelej inteligencie som použila v súlade s príslušnými právnymi predpismi, akademickými právami a slobodami, etickými a morálnymi zásadami za súčasného dodržania akademickej integrity a že ich použitie je v práci vhodným spôsobom označené.

Bratislava, 2026

.....
Bc. Anna Lenhardtová

Podakovanie

Na tomto mieste by som rada podakovala môjmu školiteľovi Ing. Lukášovi Radoskému za dôsledné vedenie, kvalitnú pomoc a cenné rady.

Abstrakt

Vo vývoji softvérových systémov sa často stretávame s opakujúcimi sa problémami, ktoré vyžadujú overené a efektívne riešenia. Softvérové inžinierstvo na tieto výzvy reaguje prostredníctvom architektonických štýlov a návrhových vzorov, ktoré ponúkajú osvedčené prístupy k organizácii štruktúry a správania systému na rôznych úrovniach abstrakcie. V tejto práci analyzujeme možnosti modelovania vybraných architektonických štýlov a návrhových vzorov — konkrétne štýlu Potrubia a filtre a návrhových vzorov Staviteľ a Most — na rovnakej úrovni granularity, ako je napríklad diagram tried. Cieľom je vytvoriť zjednotené a vykonateľné modely týchto konceptov v rámci softvérového nástroja AnimArch, ktorý umožňuje vizuálnu animáciu správania systémov. V práci doplníme katalóg systému AnimArch o nové modely a overujeme ich vhodnosť prostredníctvom simulácií a analýzy správania. Výsledkom je metodika, ktorá podporuje návrh a vizualizáciu architektúr, čo prispieva k lepšej pochopiteľnosti, modularite a udržiavateľnosti softvérových riešení.

Kľúčové slová: Architektonické štýly, návrhové vzory, Staviteľ, Most, Potrubia a filtre

Abstract

In software development, we often encounter recurring problems that require proven and efficient solutions. Software engineering addresses these challenges through architectural styles and design patterns, which offer established approaches to organizing the structure and behavior of a system at different levels of abstraction. This paper analyzes the modeling possibilities of selected architectural styles and design patterns — specifically the Pipes and Filters style and the Builder and Bridge design patterns — at the same level of granularity, such as class diagrams. The aim is to create unified and executable models of these concepts within the AnimArch tool, which allows for the visual animation of system behavior. In this work, we extend the AnimArch system’s catalog with new models and verify their suitability through simulations and behavior analysis. The result is a methodology that supports the design and visualization of architectures, contributing to better comprehensibility, modularity, and maintainability of software solutions.

Keywords: Architecture styles and patterns, Builder, Bridge, Pipes & filters

Obsah

1	Základné pojmy	2
1.1	Unifikovaný modelovací jazyk - UML	2
1.2	Jazyk objektových akcií - OAL	3
1.3	Modelom riadený vývoj - MDD	4
1.4	Návrhové vzory	5
1.5	Architektonické štýly	9
2	Modelovacie nástroje	12
2.1	AnimArch	12
2.2	Enterprise Architect	14
2.3	BridgePoint	15
2.4	Generic Modeling Environment - GME	16
3	Vzor Staviteľ	18
3.1	Návrh	18
3.2	Prvá implementácia	18
3.3	Druhá implementácia	22
4	Vzor Most	31
4.1	Návrh	31
4.2	Implementácia	31
5	Štýl Potrubia a filtre	33
5.1	Návrh	33
5.2	Prvá implementácia	33
5.3	Druhá implementácia	37
6	Evaluácia	40
7	Manuál	41

Zoznam obrázkov

1.1	Všeobecný triedny diagram vzoru Staviteľ [5]	5
1.2	Všeobecný triedny diagram vzoru Kompozit [5]	6
1.3	Všeobecný triedny diagram vzoru Abstraktná továreň [5]	7
1.4	Všeobecný triedny diagram vzoru Most [5]	8
1.5	Všeobecný triedny diagram vzoru Adaptér [5]	8
1.6	Diagram štýlu Potrubia a filtre [13]	9
1.7	Diagram štýlu Tabuľa: BB - tabuľa, KS - znalostné zdroje 1 až n [13] .	10
1.8	Diagram Viacvrstvovej architektúry [13]	10
2.1	Užívateľské prostredie nástroja AnimArch	12
2.2	Užívateľské prostredie nástroja EA, zdroj: https://sparxsystems.com/products/ea/15/	15
2.3	Užívateľské prostredie nástroja BridgePoint	16
2.4	Užívateľské prostredie nástroja GME [1]	17
3.1	Triedny diagram vzoru Staviteľ s jedným riaditeľom	19
3.2	Animácia prvej implementácie vzoru Staviteľ v nástroji AnimArch . . .	21
3.3	Triedny diagram vzoru Staviteľ	23
3.4	Sekvenčný diagram príkladu vzoru Staviteľ s uzavretým levelom	26
3.5	Sekvenčný diagram príkladu vzoru Staviteľ s otvoreným levelom	27
3.6	Animácia druhej implementácie vzoru Staviteľ v nástroji AnimArch . .	29
4.1	Triedny diagram vzoru Most	32
5.1	Prvý návrh triedneho diagramu štýlu Potrubia a Filtre	34
5.2	Druhý návrh triedneho diagramu štýlu Potrubia a Filtre	38

Zoznam tabuliek

Terminológia

Termíny

Stavový automat – model popisujúci zmeny stavov objektu v čase.

Skratky

UML	Unified Modeling Language	Unifikovaný modelovací jazyk
xUML	Executable UML	Vykonateľný UML
OAL	Object Action Language	Jazyk objektových akcií
MDD	Model-Driven Development	Vývoj riadený modelmi
MDA	Model-Driven Architecture	Architektúra riadená modelmi
PIM	Platform-Independent Model	Platformovo nezávislý model
PSM	Platform-Specific Model	Platformovo špecifický model
EA	Enterprise Architect	Modelovací nástroj Enterprise Architect
XMI	XML Metadata Interchange	Formát pre výmenu XML meta- dát
GME	Generic Modeling Environment	Modelovací nástroj Generic Mo- deling Environment
OMG	Object Management Group	Object Management Group
BB	Blackboard	Architektonický štýl Tabuľa
KS	Knowledge Source(s)	Znalostný zdroj(e)
ANTLR	ANother Tool for Language Re- cognition	Nástroj pre rozpoznanie jazyka

Motivácia

V rámci tejto práce sa zameriavame na návrh a implementáciu animácií vybraných návrhových vzorov a architektonického štýlu. Vzory a štýly patria v praxi medzi zaužívané pojmy, ktoré každý softvérový inžinier potrebuje poznať. Veríme, že vizualizácie našich príkladov by mohli pomôcť študentom jednoduchšie porozumieť vybraným vzorom a štýlu. Konkrétne ich vlastnostiam a spôsobu, akým sa ich jednotlivé komponenty správajú a spolupracujú.

Kapitola 1

Základné pojmy

V tejto kapitole vysvetlíme všetky pojmy spojené s tvorbou systému, ako sú UML diagramy, zaužívané postupy, návrhové vzory a architektonické štýly.

1.1 Unifikovaný modelovací jazyk - UML

Unifikovaný modelovací jazyk (angl. Unified Modeling Language), ďalej označovaný UML, je grafický jazyk používaný na vizualizáciu, špecifikáciu, konštrukciu a dokumentáciu artefaktov informačných systémov [2]. Pozostáva z rôznych typov diagramov, ktoré delíme na dve základné skupiny: **štrukturálne** a **behaviorálne**. Okrem nich opíšeme aj osobitnú skupinu diagramov - **Vykonateľné UML**.

Štrukturálne diagramy

Štrukturálne diagramy reprezentujú štruktúru alebo statiku systému či jeho podčasti. Zobrazujú jednotlivé entity vystupujúce v systéme a prepojenia medzi nimi.

Triedny diagram je jedným zo základných štrukturálnych diagramov. Jeho entity predstavujú triedy alebo rozhrania vybranej časti systému. Zobrazujeme ich v obdĺžnikových rámkoch s názvom triedy, triednych atribútov a metód. Vzťahy medzi entitami sa určujú čiarami so šípkami. Podľa tvaru šípky rozlišujeme dedenie (čiara s plnou šípkou), agregáciu (čiara s kosoštvorcom na strane celku), alebo asociáciu (čiara bez šípky alebo s jednoduchou šípkou) medzi dvomi triedami, ktoré spájajú.

Objektový diagram zobrazuje objekty, teda vytvorené inštancie tried, prepojenia medzi nimi a hodnoty ich atribútov. Od triedneho diagramu sa líši tým, že každý objekt má definovanú triedu a môže zobrazovať viac inštancií jednej triedy.

Behaviorálne diagramy

Behaviorálne diagramy reprezentujú interakciu alebo dynamiku systému či jeho podčasti. Ukazujú komunikáciu medzi entitami alebo zmeny v priebehu času.

Sekvenčný diagram je behaviorálny diagram, ktorý zobrazuje posielanie správ medzi entitami a ich spätnú reakciu na ne, poprípade vyvolávanie metód v ďalšej entite, podľa zvolenej úrovne abstrakcie. Na vrchnej (a niekedy aj spodnej) časti diagramu sú jednotlivé entity. Každá entita má priradenú vertikálnu prerušovanú čiaru (angl. lifeline), na ktorej sú zobrazené ich aktivity pomocou aktivačných blokov. Medzi týmito čiarami sú šípky podľa toho, ktorá entita komunikuje s ktorou a aj v akom okamihu. Vertikálna os diagramu smerom dole predstavuje prebiehajúci čas.

Stavový diagram reprezentuje zmenu stavu konkrétnej entity podľa zmien v systéme. Skladá sa zo stavov v obdĺžnikoch prepojených šípkami s opisom zmeny, ktorá keď nastane, stav entity bude zodpovedať stavu na ktorý šípka ukazuje.

Vykonateľné UML - xUML

Vykonateľné UML (angl. Executable Unified Modeling Language, xUML) sú modely, ktoré kombinujú UML grafickú notáciu so sémantikou vykonávania. Modely xUML je možné spustiť, simulovať a testovať a dá sa z nich priamo generovať zdrojový kód. Tento prístup podporuje platformovo nezávislé modelovanie.

Základom xUML sú domény, triedne diagramy, stavové automaty a jazyk akcií. Domény predstavujú logické oblasti systému, ktoré umožňujú oddelenie zodpovedností a uľahčujú modelovanie jednotlivých častí systému [9]. Triedne diagramy popisujú štruktúru systému – triedy, ich atribúty a asociácie. Stavové automaty definujú správanie jednotlivých inštancií tried v čase, pričom prechodom medzi stavmi sú priradené konkrétne akcie. Akcie sa v xUML zapisujú v jednoduchom jazyku **OAL** (pozri 1.2), ktorý umožňuje manipuláciu s inštanciami tried a ich vzťahmi [15].

Hlavnou výhodou xUML je možnosť opätovného použitia modelov naprieč rôznymi platformami a jazykmi. Modely vytvorené ako PIM (pozri 1.3) môžu byť preložené do rôznych implementačných jazykov pomocou vhodných kompilátorov. Týmto spôsobom xUML podporuje efektívny a jednotný vývoj komplexných softvérových systémov.

1.2 Jazyk objektových akcií - OAL

Jazyk objektových akcií (angl. Object action language), ďalej označovaný OAL, je jazyk používaný pri vytváraní modelov xUML. Predstavuje vykonateľnú stranu modelov. Kompletný manuál poskytuje na webovej stránke spoločnosť BridgePoint¹. V nasledu-

¹Dostupné na: <http://www.oatool.com/docs/OAL08.pdf>

júcej podkapitole opíšeme základnú syntax jazyka OAL.

Syntax OAL

Jednotlivé príkazy jazyka sú oddelené bodkočiarkou, až na zložitejšie konštrukcie ako cykly a podmienky. OAL nerozlišuje pri kľúčových slovách veľké a malé písmená. Tu je výber niektorých kľúčových slov: *break*, *class*, *continue*, *create*, *delete*, *elif*, *else*, *end*, *false*, *for*, *if*, *instance*, *not*, *object*, *of*, *return*, *self*, *true*, *while*. Za komentár sa považuje čokoľvek za znakmi `"/"` do konca riadku [10].

Všetky dátové prvky musia mať zadefinovaný typ, a to implicitne podľa typu priradenej hodnoty. OAL rozlišuje napríklad tieto typy: *integer*, *string*, *real*, *date*, *boolean*, *timestamp*. Premenné, ktoré nemajú priradenú hodnotu sú považované za nedefinované ("UNDEFINED") [10].

V jazyku funguje klasické priradenie do premennej znakom `"="` a porovnávanie medzi dvomi prvkami `"!="`, `"=="`, `">"`, `"<"`, `"<="`, `">="`. Vo výrazoch sa používajú logické operátory `"and"`, `"or"` a `"not"` ako konjunkcia, disjunkcia a negácia [10].

Konštrukcie `"while"` a `"if"` sa začínajú kľúčovým slovom nasledované výrazom v zátvorke, ktoré nie je zakončené bodkočiarkou (napr. `"while (index < 4)"`). Cyklus `"for each"` má syntax: `"for each <pomenovanie> in <kolekcia>"`. V ďalších riadkoch je obsah cyklu alebo podmienky. Ak sa jedná o podmienku, môže blok kódu obsahovať ešte riadky: `"elif (výraz)"` alebo `"else"`. Tieto taktiež nie sú zakončené bodkočiarkou. Blok je zakončený kľúčovým slovom `"end"`, kľúčovým slovom konštrukcie a bodkočiarkou (napr. `"end while;"` alebo `"end for;"`). Konštrukcie môžu byť vnorené [10].

Kľúčové slovo `"break"` spôsobí vyskočenie z cyklu a slovo `"continue"` spôsobí nasledujúcu iteráciu cyklu [10].

Vytváranie inštancií triedy majú konštrukciu: `"create object instance <pomenovanie> of <trieda>"`. Na druhú stranu na vymazanie inštalácie sa použije syntax: `"delete object instance <pomenovanie>"` Prístup ku parametrom a vyvolaniu metód inštalácie triedy je definovaný bodkou, napr. `"self.attribute"` alebo `"self.method()"`.

1.3 Modelom riadený vývoj - MDD

Modelom riadený vývoj (skratka MDD, angl. "Model driven development") je prístup vývoja softvéru, ktorý používa ako základ modely a modelovacie technológie. Modely slúžia ako jednotný zdroj dokumentácie [6]. Cieľom je z modelov automaticky generovať zdrojový kód [3].

Častou MDD je Modelom riadená architektúra (skratka MDA, angl. "Model driven Architecture") vytvorené skupinou Object Management Group (skratka OMG). Základom je Platformovo nezávislý model (skratka PIM, angl. "Platform-Independent

Model”), ktorý popisuje funkčnosť systému bez technologických detailov ako celok, a Platformovo špecifický model (skratka PSM, angl. ”Platform-Specific Model”), ktorý špecifikuje implementáciu pre konkrétnu platformu [6]. Dopĺňa PIM o detaily jedinečné pre technologické prostredie, na ktorom bude systém nasadený alebo implementovaný, ako programovací jazyk, typ databázy, operačný systém a pod.

1.4 Návrhové vzory

Návrhový vzor je znovupoužiteľná šablóna, ktorá rieši konkrétny problém v objektovo-orientovanom programovaní. Podľa [5] má návrhový vzor 4 hlavné časti: meno, problém, riešenie a následky.

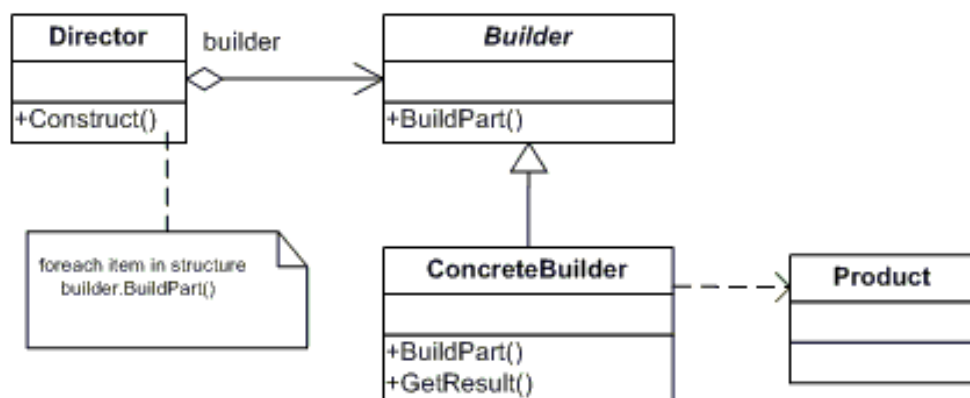
Meno vzoru ho pomáha identifikovať nie len ako nejaký termín v dokumentácii, ale hlavne ako názov, ktorý sa aktívne používa pri vývoji. Každý vie, čo sa skrýva pod daným pojmom a nie je potrebné ho vysvetľovať počas diskusie v tíme, keď sa rieši nejaká dilema.

Problém vysvetľuje kontext vzoru, teda v akom prípade je vhodné ho použiť. Niekedy je definovaný ako zoznam podmienok, ktoré ak nastanú, je vhodné vzor aplikovať.

Riešenie všeobecne opisuje komponenty, ako by mali vyzeráť ich vzťahy, zodpovednosti a vzájomná spolupráca. Je skôr ako návod, podľa ktorého môžeme postupovať.

Následky pomáhajú určiť dopad použitia vzoru na systém, čo sa týka napríklad priestoru, ktorý zaberá, alebo času, aký využíva. Uľahčujú rozhodovanie, ktorý vzor použiť, ak je viacero alternatív riešenia daného problému.

Prioritne sa venujeme vzorom Staviteľ a Most, okrem nich priblížime aj niektoré ich príbuzné návrhové vzory.

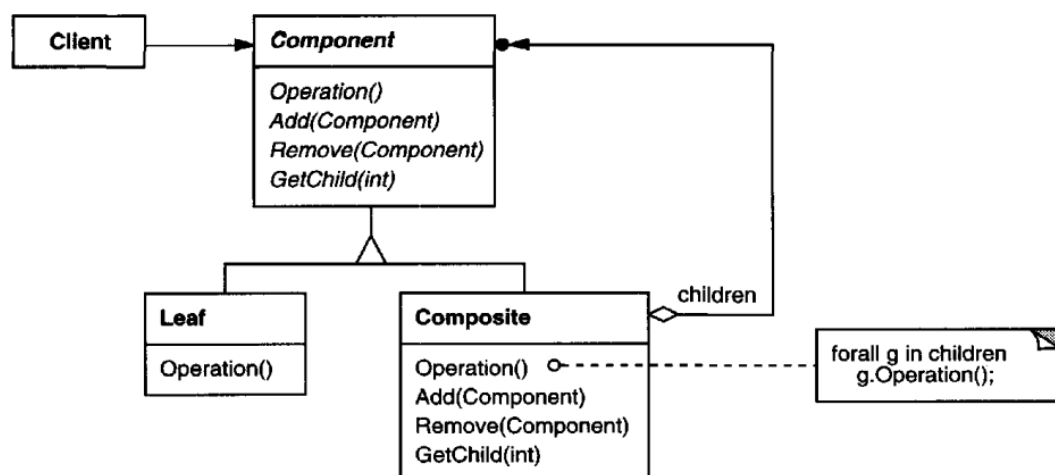


Obr. 1.1: Všeobecný triedny diagram vzoru Staviteľ [5]

Hlavná myšlienka vzoru **Staviteľ** (angl. Builder) je podľa [5] to, že oddeľuje kon-

štrukciu komplexného objektu od jeho reprezentácie, aby mohol rovnaký proces konštrukcie vytvoriť jeho rôzne reprezentácie. Alebo naopak aby bol výsledný objekt rôzne zostavený v rovnakej reprezentácii. Používa sa vtedy, keď je potrebné, aby bolo vytvorenie celého komplexného objektu nezávislé od tvorby jeho jednotlivých častí a spôsobu ich zostavenia.

Na obrázku 1.1 je vidieť, že vzor pozostáva z častí: Riaditeľ (angl. Director), Abstraktný staviteľ (angl. Abstract builder), Konkrétny staviteľ (angl. Concrete builder) a Produkt (angl. Product). Riaditeľ obsahuje postup, akým má byť komplexný objekt postavený vybraným staviteľom. Obsahuje volania metód triedy Konkrétneho staviteľa v určitom poradí. Väčšinou je tento vzor implementovaný s viacerými riaditeľmi, ktoré dedia od abstraktného riaditeľa. Každý z nich má spravidla iný postup na zostavenie výsledku. Abstraktný staviteľ je trieda od ktorej môžu dediť všetky konkrétne triedy staviteľov, ale vďaka abstraktnej triede majú preddefinované rovnaké metódy. Konkrétny staviteľ implementuje všetky potrebné metódy na zostavenie výsledného objektu. Produkt predstavuje výsledok.

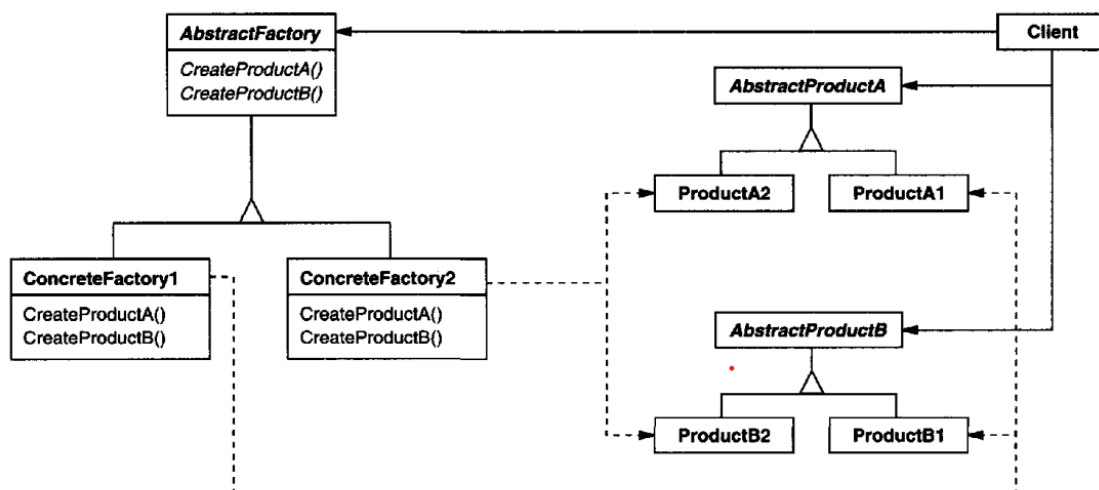


Obr. 1.2: Všeobecný triedny diagram vzoru Kompozit [5]

Kompozit (angl. Composite) je vzor asociovaný so stromovou štruktúrou. Jeho triedny diagram je zobrazený na obrázku 1.2. Skladá sa z abstraktnej triedy Komponent (angl. Component), od ktorej dedia triedy List (angl. Leaf) a Kompozit (angl. Composite). List je ako v klasickej stromovej štruktúre prvok bez detí. Kompozit má deti typu Komponent, teda môžu to byť opäť inštancie typu List alebo Kompozit. Hlavnou zložkou tohto vzoru je metóda na spracovanie prvkov stromu, ktorú obsahujú všetky triedy a podľa typu je implementovaná - ukončuje "rekurziu" alebo vyvoláva rovnakú metódu na svojich deťoch.

Staviteľ sa spája s refaktorovaním zo vzoru Kompozit. Pri vzore Kompozit validuje, vytvára a upravuje stromovú štruktúru Klient. Kerievsky [7] navrhuje abstrahovať tieto

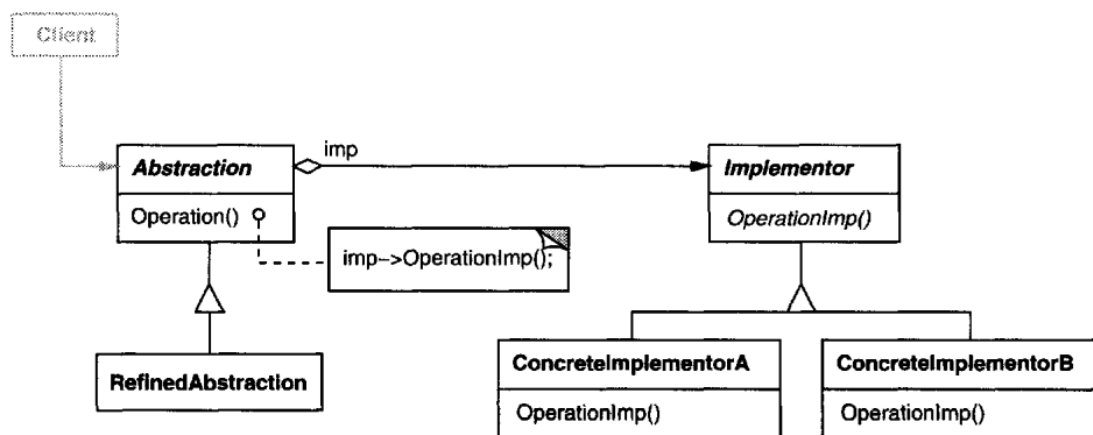
postupy pomocou Staviteľa, aby klient nemusel vykonávať každý krok, ako je potrebné v objektovo orientovanom programovaní. Sú to napríklad vytváranie inšancie vrcholu, jeho priradenie ako parametra rodičovskému vrcholu a pod. Staviteľ všetky tieto kroky vykoná podľa požiadaviek klienta.



Obr. 1.3: Všeobecný triedny diagram vzoru Abstraktná továreň [5]

Vzor **Abstraktná továreň** (angl. Abstract factory) je podobný vzoru Staviteľ v tom, že vytvára konkrétny objekt podľa požiadaviek. Líši sa v tom, že poskytuje možnosť vytvorenia všetkých kombinácií objektov patriacich do rôznych rodín (N rodín x M objektov možností) a v konečnom dôsledku nevytvára len jeden komplexný produkt, ale niekoľko produktov z jednej rodiny. V zásade sa používajú v jednom systéme produkty z jednej rodiny, lebo tie sú navzájom kompatibilné, poprípade vedia spolupracovať.

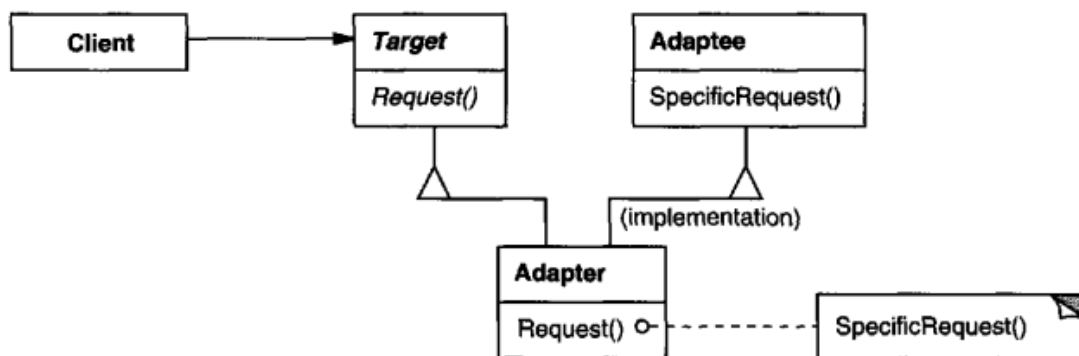
Ako je zobrazené na obrázku 1.3, skladá sa z častí: Abstraktná továreň, Konkrétna továreň (angl. Concrete factory), Abstraktný produkt (angl. Abstract product), Konkrétny produkt (angl. Concrete product) a Klient. Abstraktná továreň predpisuje štruktúru konkrétnych tovární, ich atribúty a metódy. Konkrétna továreň zodpovedá za vytváranie produktov konkrétnej rodiny objektov. Abstraktný produkt definuje štruktúru produktu a od tejto triedy dedia Konkrétne produkty z každého typu rodiny [5].



Obr. 1.4: Všeobecný triedny diagram vzoru Most [5]

Most (angl. Bridge) rozdeľuje abstrakciu a implementáciu do dvoch hierarchií tried, aby sa vyvíjali nezávisle [5]. Využíva sa vtedy, keď implementujeme rovnaké funkcionality pre viaceré platformy alebo prostredia. Namiesto toho, aby sme vytvárali triedy pre všetky kombinácie funkcionalít a platforiem, vytvárame hierarchiu, v ktorej každej funkcionalite zodpovedá viacero implementácií (podľa platformy). Potom využitie danej funkcionality je platformovo nezávislé. Klient pracuje iba s abstrakciou a platforma je definovaná na inom mieste.

Ako je vidieť na obrázku 1.4, pozostáva z dvoch hierarchií: trieda Abstrakcia (angl. Abstraction), od ktorej dedí trieda Refinovaná abstrakcia (angl. RefinedAbstraction) a na druhej strane trieda Implementátor (angl. Implementor), od ktorej dedia konkrétne implementátory A a B (angl. ConcreteImplementorA a ConcreteImplementorB). Abstrakcia je celok agregácie implementátora, teda drží referenciu na konkrétneho implementátora [5].



Obr. 1.5: Všeobecný triedny diagram vzoru Adaptér [5]

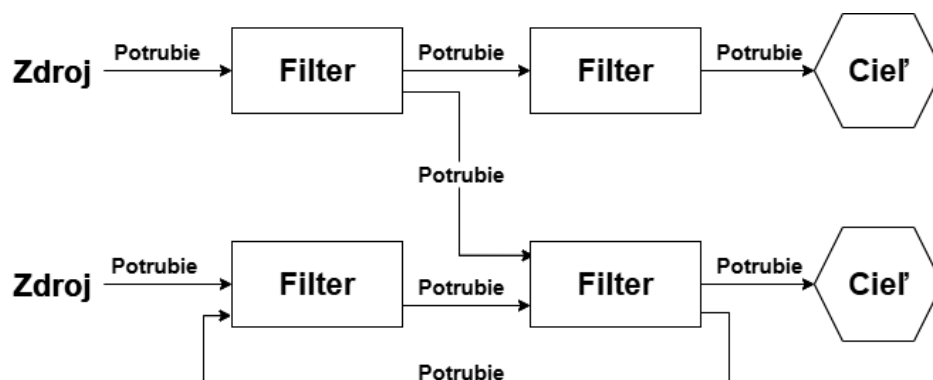
Adaptér je vzor, ktorý sa používa, keď klient očakáva isté rozhranie, ale používaná trieda má iné rozhranie. Do hierarchie je pridaná trieda Adaptér, ktorá sprostredkuje klientovi cez známe rozhranie funkcie tejto triedy. Trieda Adaptér môže byť imple-

mentovaná ako trieda, ktorá dedí od triedy Prispôsobovaný objekt alebo ju má ako referenciu [5].

Na obrázku 1.5 sú znázornené triedy vzoru Adaptér pri implementácii Adaptéra ako podtriedy: Klient, Cieľ (angl. Target) je trieda, ktorú používa klient, ja nadtriedou triedy Adaptér. Adaptér sa správa ako trieda, ktorú pozná klient, ale vyvoláva metódy na triede Prispôsobovaný objekt (angl. Adaptee). V nej sa skutočne implementuje požadovaná funkcionálna 1.5.

1.5 Architektonické štýly

Pri vývoji systému slúžia architektonické štýly na organizáciu komponentov tohto systému. Existuje niekoľko štýlov, ktoré pomáhajú modelovať napríklad zdieľanie pamäte, distribúciu, posielanie správ, celkovú štruktúru a iné [13]. V nasledujúcich odstavcoch sa budeme venovať niektorým z nich.



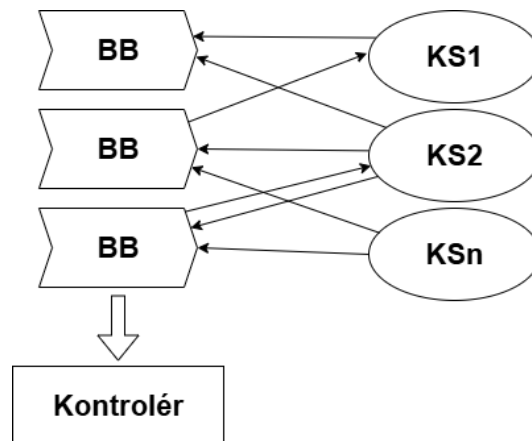
Obr. 1.6: Diagram štýlu Potrubia a filtre [13]

Potrubia a filtre (angl. Pipes and filters) je architektonický štýl, ktorý opisuje štruktúru systému. Už z názvu je zrejmé, že pozostáva z dvoch základných častí. Filter je komponent, ktorý spracováva vstup a potrubie prepája filtre a presúva medzi nimi dáta. Prvé vstupné potrubie sa označuje ako zdroj (angl. pump alebo source) a posledné výstupné ako cieľ (angl. sink) [13].

Zdroj pošle vstupné dáta prvému filtru, ktorý ich spracuje a pripraví na ďalšie spracovanie. Výhodou je, že ak je filter hotový, je pripravený spracovávať ďalší vstup. Filtre môžu byť rôzne usporiadané a prepojené, teda môžu vytvárať cyklus alebo vetvenie na viaceré filtre podľa výstupu. Použitie tejto štruktúry má výhody v znovupoužití a preusporiadaní filtrov a potrubí, podporuje paralelizmus. Nevýhodou je náročné spracovanie chyby, keďže sa stavy medzi filtrami nezdieľajú. Diagram tohto štýlu je zobrazený na obrázku 1.6.

Spôsob vykonávania v tejto architektúre je v zásade označovaný ako "pull" alebo "push". "Pull" reprezentuje ťahanie, teda potrubie si žiada vykonávanie od predchá-

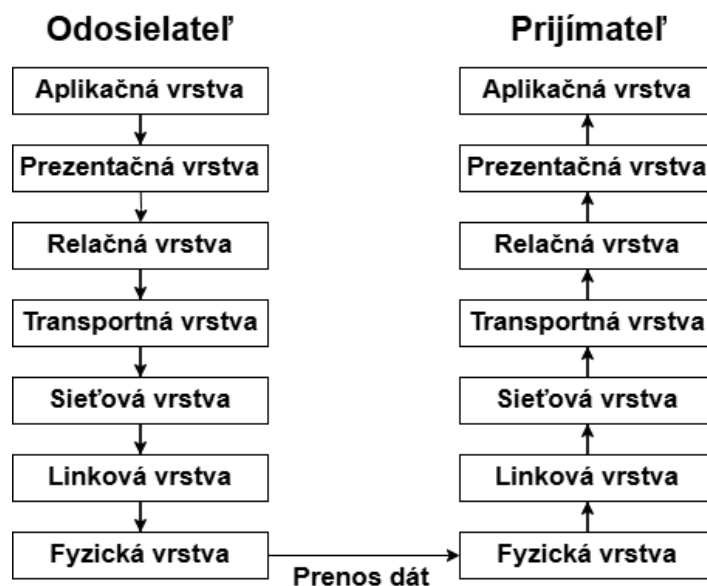
dzajúceho filtra a "push" tlačenie od nasledujúceho filtra.



Obr. 1.7: Diagram štýlu Tabuľa: BB - tabuľa, KS - znalostné zdroje 1 až n [13]

Tabuľa (angl. Blackboard) je architektonický štýl, ktorý rieši nedeterministické problémy a to takým spôsobom, že ako repozitár zdieľa daný problém. Komponenty nazývané **znalostné zdroje** (angl. knowledge sources) vidia obsah repozitára a reagujú na podnety, ktoré dokážu vyriešiť. Každý zdroj má nejakú funkcionality, ktorou vie upraviť repozitár. V ideálnom prípade sa postupnou úpravou dát v repozitári dostávame bližšie k výsledku.

Nevýhodou štýlu je, že je náročné určiť, kedy sme dosiahli riešenie a taktiež, postarať sa o synchronizáciu viacerých zdrojov. Na druhej strane je štýl ľahko rozširiteľný alebo redukovateľný, čo sa týka zdrojov [13]. Jeho diagram je na obrázku 1.7.



Obr. 1.8: Diagram Viacvrstvovej architektúry [13]

Viacvrstvová architektúra (angl. Layered style) je architektúra, ktorá rozdeľuje systém do niekoľkých voľne viazaných vrstiev na základe ich zodpovedností. Vrstvy

komunikujú v dobre definovanom rozhraní. Rozlišujeme tri základné typy vrstiev: prezentačná, biznisová a infraštruktúrna [13]. Rozložený aj na viac vrstiev ho môžete vidieť na obrázku 1.8.

Prezentačná vrstva zaručuje komunikáciu užívateľa so systémom cez užívateľské rozhranie. Biznisové vrstva predstavuje samotnú logiku systému. Infraštruktúrna vrstva zabezpečuje dátové, sieťové a iné podporné služby systému [13].

Kapitola 2

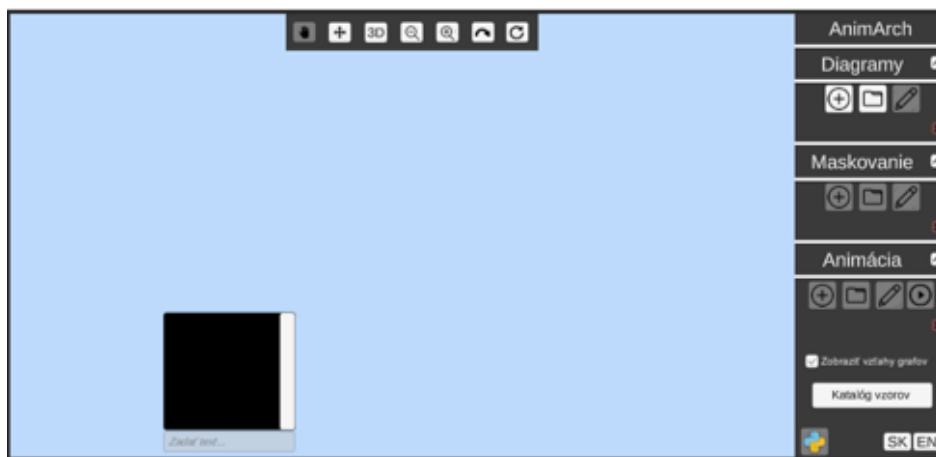
Modelovacie nástroje

V tejto kapitole bližšie opíšeme modelovacie nástroje, ktoré sa v súčasnosti používajú pri MDD. Zameriavame sa na ich vlastnosti, funkcionality a užívateľské prostredie.

2.1 AnimArch

AnimArch je aplikácia vytvorená v Unity v jazyku C#. Slúži na vizualizáciu a animáciu kódu. Je postavený na statickej a dynamickej zložke: triedny diagram a kód napísaný v modifikovanej syntaxi OAL [11].

Triedny diagram je možné upravovať priamo v AnimArch-u alebo nahrať ako súbor formátu XMI 2.1. OAL kód môže užívateľ pridať do jednotlivých metód tried v triednom diagrame alebo si zvoliť existujúci súbor s kompatibilným OAL kódom, ktorý už bol napísaný a uložený. Súbor predstavujúci systém (triedny diagram) a súbor s OAL sa vyberajú samostatne, takže pre jeden triedny diagram môžeme vybrať viacero k nemu kompatibilných animácií [11].



Obr. 2.1: Užívateľské prostredie nástroja AnimArch

Na obrázku 2.1 je zobrazené užívateľské prostredie AnimArch-u. Na pravej strane si

užívateľ môže vybrať z už existujúceho súboru, vytvoriť nový alebo dodatočne upraviť triedny diagram, maskovanie a animáciu. Animáciu je možné navyše spúšťať. Okrem toho je možné generovať kód v jazyku Python kliknutím na ikonku tohto jazyka. Tiež zmeniť jazyk nástroja ako takého. Na ľavej strane sa nachádza konzola, ktorá slúži na vstupy od užívateľa a textové výpisy spúšťaného kódu. V hornej lište s v danom poradí môže užívateľ posúvať diagramy tromi spôsobmi, približovať, vzdalovať diagram, presúvať sa medzi vrstvami a zobrazíť priamo prednú vrstvu.

Užívateľ si zvolí konkrétnu metódu v niektorej triede na zobrazenom triednom diagrame, ktorú chce animovať. Ak nie je táto počiatočná metóda prázdna, spustí sa parsovanie kódu cez ANTLR do abstraktného syntaktického stromu. Po spustení sa vygenerujú za triednym diagramom do 3D priestoru ďalšie UML diagramy: triedny diagram, diagram objektov a diagram aktivít.

Animácia prebieha v grafickom rozhraní AnimArch-u a to tak, že zároveň s postupným vykonávaním príkazov sa zvýrazňujú triedy a vzťahy (šípky), s ktorými súvisia, a vytvárajú inštalácie v diagrame objektov v druhej vrstve. V tretej vrstve na diagrame aktivít sa zvýrazňujú vykonávané príkazy, podmienky a pod.

Užívateľ môže pozorovať v nastaviteľnej rýchlosti následnosť spúšťaného kódu, vytváranie potrebných inšancií a to, do ktorej triedy a metódy kód patrí. Navyše môže zvolený OAL kód exportovať do plne funkčného kódu v jazyku Python.

V AnimArch-u sme do importovaných triednych diagramov vzorov a štýlu dopĺňali OAL kódy a následne sme ich vykonávanie vizualizovali a animovali.

OAL prispôbené v AnimArch-u obsahuje aj nasledujúce metódy na type reťazec, ktoré boli využité pri písaní kódu pre animácie vzorov a štýlov:

- **string Join(string[] joinedElements)**: vytvorí jeden reťazec spojením prvkov poľa, pričom reťazec, nad ktorým je metóda volaná je vložený medzi jednotlivé prvky;
- **string[] Split(string oddelovač)**: rozdelí pôvodný reťazec na pole podreťazcov podľa zadaného oddelovača, ktorý určuje miesto delenia v texte;
- **integer FirstIndexOf(string podreťazec)**: vráti index prvého výskytu podreťazca v reťazci;
- **integer[] AllIndexesOf(string podreťazec)**: vráti pole indexov so všetkými výskytmi podreťazca v reťazci;
- **integer Length()**: vráti dĺžku reťazca;
- **string SubstringFrom(integer štartIndex)**: vráti časť reťazca od štartIndex-u po koniec reťazca;

- **string Substring(integer štartIndex, integer počet):** vráti reťazec od štartIndex-u dĺžky počet;
- **boolean Contains(string podreťazec):** vyhodnotí ako true alebo false podľa toho, či sa podreťazec nachádza v reťazci;
- **string Replace(string starý, string nový):** vráti reťazec, v ktorom nahradí všetky výskyty reťazca starý za reťazec nový.

OAL v AnimArch-u navyše definuje vytváranie prázdneho poľa konštrukciou: "create list <pomenovanie> of <typ>". Prístupovanie k prvkom poľa cez index je možné cez hranaté zátvorky, napr.: pole[0]. Pri používaní polí sme využili najmä metódu **Count()**. OAL poskytuje okrem nej ešte ďalšie dve metódy na prácu s poľami:

- **integer Count():** vráti počet prvkov poľa;
- **integer IndexOF(typ element):** vráti index daného elementu v poli, typ závisí od typu, ktorý má definované pole;
- **boolean Contains(typ element):** vyhodnotí to, či sa element nachádza v poli, ako pravdivé alebo nepravdivé tvrdenie, typ musí byť opäť zhodný s typom poľa.

Ďalej OAL v AnimArch-u umožňuje prácu so súbormi. Cestu k umiestneniu súboru je potrebné definovať absolútne. Metódy:

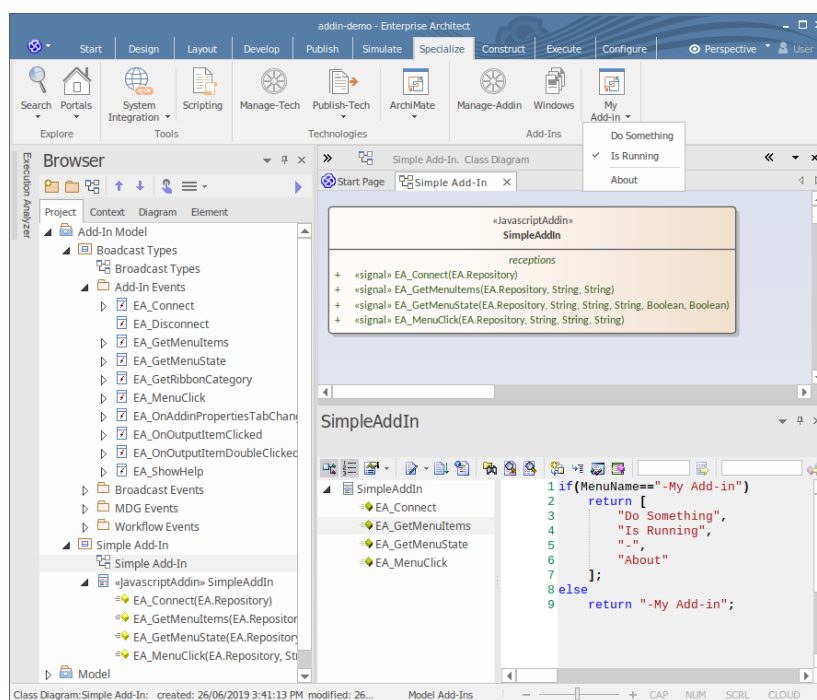
- **string ReadFile(string cesta):** prečíta obsah súboru z cesty, ktorý je možné priradiť do premennej;
- **WriteToFile(string cesta, string obsah):** zapíše obsah do súboru na ceste.

2.2 Enterprise Architect

Enterprise Architect, ďalej ako EA, je modelovací nástroj od spoločnosti Sparx Systems na vytváranie UML diagramov potrebných pre vývoj softvérových systémov. Taktiež sa používa na kolaboráciu a zdieľanie pri tvorbe vo väčších tímoch [14].

EA podporuje všetky typy UML diagramy, užívateľ si môže vybrať štrukturálne aj behaviorálne diagramy. V interaktívnom prostredí je možné pridávať komponenty a upravovať vzťahy medzi nimi. V EA sme vytvárali triedne diagramy vzorov a štýlu. Triedy sme mohli ľubovoľne upravovať, pridávať triedne atribúty s typom a metódy s parametrami, tiež rôznych typov. Pridávanie vzťahov medzi triedami je intuitívne a konkrétne pri dedení dokáže EA reagovať automatickým prepísaním metód z rodičovskej triedy do zdedenej triedy. Jednotlivé metódy a triedy majú viaceré vlastnosti, ako

abstraktnosť a pod. Rozloženie diagramu si môže užívateľ taktiež prispôbiť. Keď bol diagram hotový, exportovali sme ho do súboru vo formáte XMI 2.1. Diagram je možné exportovať aj ako obrázok formátu png.



Obr. 2.2: Užívateľské prostredie nástroja EA, zdroj: <https://sparxsystems.com/products/ea/15/>

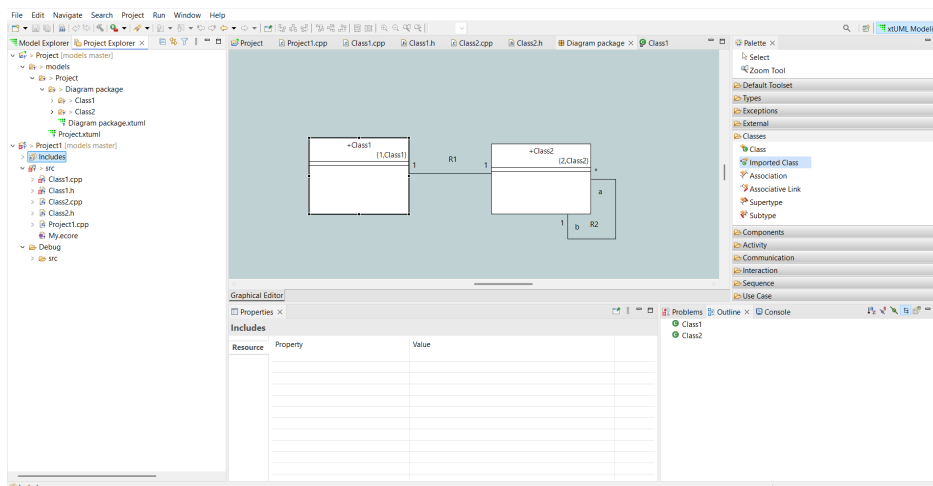
Na obrázku 2.2 je zobrazených pár z mnohých okien, v ktorých je možné pracovať v nástroji EA. V hornej lište sú viaceré záložky s nástrojmi. Na ľavej strane sa väčšinou nachádza okno so štruktúrou diagramu. V strede je zobrazený samotný diagram. Dole na obrázku sú ďalšie pridané funkcionality, ako práca s metódami triedy a ich kódom.

2.3 BridgePoint

Ďalším nástrojom používaným v MDD je BridgePoint od spoločnosti Bridgepoint Group plc. Jeho hlavným zámerom je generovanie kódu z diagramu, zaoberá sa teda vykonateľným UML (xUML). Modelovanie systému ako celku zvláda organizáciou do komponentov a aj vytvorením subsystémov v rámci jedného komponentu [15].

Na obrázku 2.3 je zobrazené užívateľské prostredie nástroja BridgePoint. Kľúčový je diagram v strede, v ktorom môže užívateľ presúvať a inak upravovať komponenty. Na ľavej strane sa nachádza štruktúra súborov a na pravej komponenty a vzťahy, ktoré môže užívateľ pridávať do diagramu. Okrem toho má k dispozícii konzolu a vlastnosti jednotlivých komponentov.

Modelovaný triedny diagram musí spĺňať charakteristiku PIM, aby z neho bolo možné generovať kód v ľubovoľnom programovacom jazyku. Najdôležitejšie sú asociácie



Obr. 2.3: Uživatelské prostredie nástroja BridgePoint

medzi triedami. Správanie systému je definované cez stavové automaty a akcie v jazyku OAL [15].

BridgePoint poskytuje aj prostredie na testovanie už počas fázy modelovania. Nástroj "Verifier" vykonáva OAL kód, vizualizuje prechody stavov, monitoruje hodnoty premenných a umožňuje samostatnú simuláciu a testovanie komponentov [15].

BridgePoint podporuje aj integráciu s externým alebo ručne písaným kódom. Ten môže byť vynechaný z generovaného kódu. Samotný proces generovania kódu je riadený pomocou značiek (angl. "marks"), ktoré určujú prepojenia komponentov, dátové typy a inicializáciu pamäte. Okrem toho poskytuje štatistiku modelu vrátane počtu tried, funkcií, stavových automatov a príkazov OAL [15].

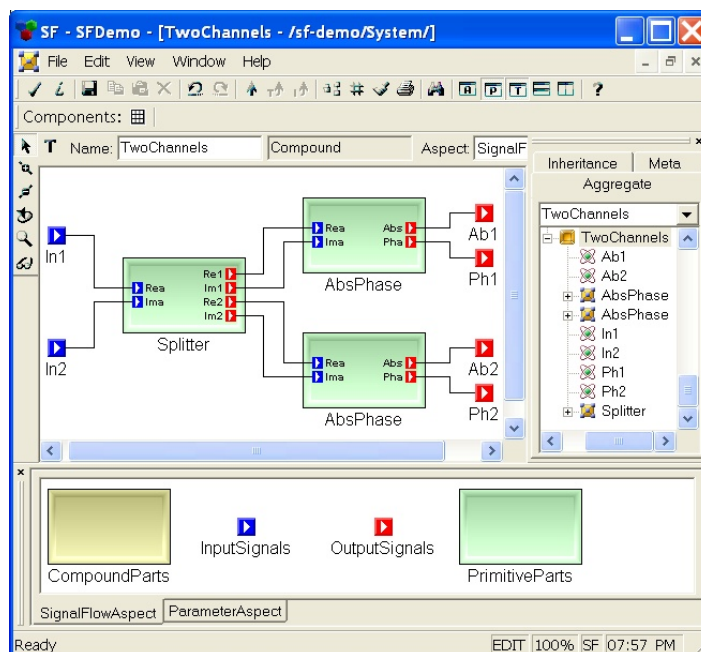
2.4 Generic Modeling Environment - GME

Generic Modeling Environment (GME) je nástroj na tvorbu doménovo špecifických modelov a automatické generovanie kódu alebo vstupov pre analytické nástroje. Používa grafické rozhranie na modelovanie systémov spolu s ich hardvérovým a softvérovým prostredím [8].

Na obrázku 2.4 je znázornené uživatelské prostredie tohto nástroja. Zobrazuje model grafu tokov signálu [1]. Na pravo sa nachádza hierarchia súborov a spodné okno zobrazuje všetky časti, ktoré sú práve používané v modeli [1].

GME funguje na základe metamodelov, ktoré definujú pravidlá a koncepty pre konkrétnu doménu. Podľa týchto metamodelov sa vytvorí prostredie, v ktorom používateľ vytvára modely. Modely môžu byť hierarchické, obsahovať jednoduché objekty (atómy) aj zložité štruktúry (modely) a aj byť vzájomne prepojené referenciami alebo množinami [8].

Základné koncepty GME zahŕňajú hierarchiu modelov, viacnásobné aspekty pre



Obr. 2.4: Uživatelské prostredie nástroja GME [1]

kontrolu viditeľnosti objektov, ukazovatele na iné objekty, množiny na zoskupovanie objektov a explicitné obmedzenia vzťahov [8].

Kapitola 3

Vzor Staviteľ

V tejto kapitole sa venujeme návrhu a implementácii príkladu pre návrhový vzor Staviteľ.

3.1 Návrh

Vzor Staviteľ rieši zostavenie zložitejšej štruktúry podľa požiadaviek klienta. Podrobnejšie sme ho už opísali v podkapitole 1.4.

Ako príklad navrhujeme **generovanie prostredia pre hru** situovanú v prírode. Konkrétny vzhľad prostredia je výsledným komplexným produktom, ktorý môže Staviteľ postaviť a vrátiť. Obsahuje typ terénu, vhodné druhy zvierat a rastlín. Užívateľ si môže vybrať z troch rôznych biómov (vzhľadov): púšť, tundra a les. Práve touto časťou navrhujeme začať proces implementácie Staviteľa. Opíšeme ho v podkapitole Prvá implementácia. Po vyhodnotení navrhujeme pridať možnosť zvoliť si typ hry popísaný v časti Druhá implementácia. Typ hry zahŕňa otvorený svet bez stien, kde sa môže postava ľubovoľne pohybovať, alebo malé prostredie ohraničené stenami, napríklad pre potreby úrovňovej hry.

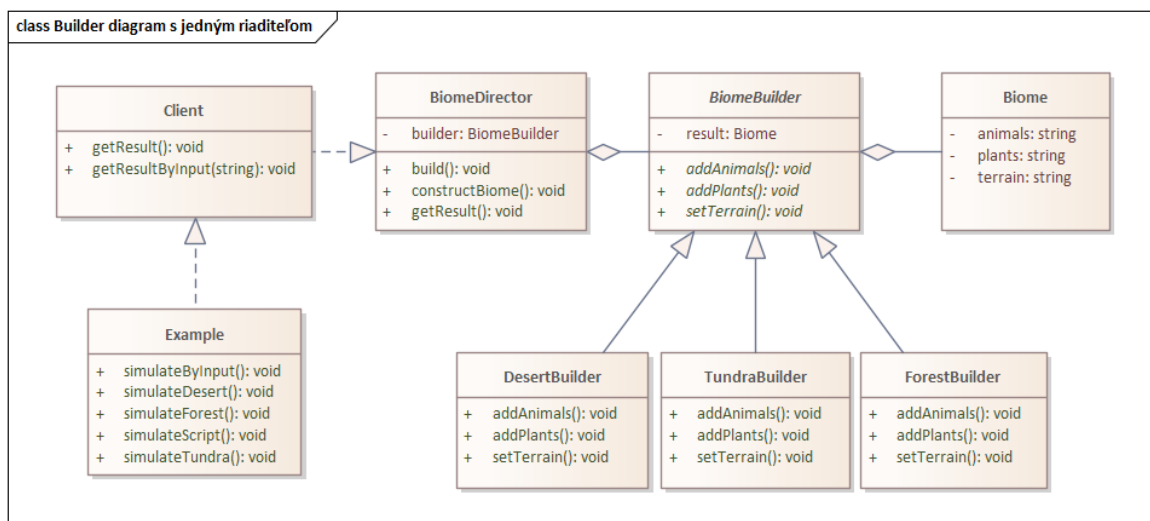
Študenti majú väčšinou dobrú predstavu o hrách a ich tvorbe, preto by pre nich náš príklad mohol byť obzvlášť zrozumiteľný. Zároveň zobrazuje využitie vzoru Staviteľ tak, ako by mohol byť reálne pri vývoji vhodnej hry použitý.

3.2 Prvá implementácia

V ďalších podkapitolách sa budeme venovať prvej implementácii vzoru Staviteľ.

Triedny diagram

Triedny diagram sme vytvorili v nástroji Enterprise Architect. Zobrazený na obrázku 3.1. Pozostáva z abstraktných tried **BiomeDirector** a **BiomeBuilder**. Od triedy



Obr. 3.1: Triedny diagram vzoru Staviteľ s jedným riaditeľom

BiomeBuilder dedia štruktúru traja konkrétny Staviteľia: **ForestBuilder**, **DesertBuilder** a **TundraBuilder**, ktoré generujú terén, zvieratá a rastliny prislúchajúce k danému biómu les, púšť alebo tundra. **Biome** je trieda, ktorá predstavuje výsledný produkt. Zahŕňa zvieratá, rastliny a terén. Okrem spomenutých tried sme zahrnuli aj triedu, ktorá predstavuje klienta a triedu **Example**, ktorá sa používa na spustenie simulácií a získanie vstupov od užívateľa.

OAL kód

Jednotlivé metódy doplníme o funkcionality vo forme OAL kódu v nástroji AnimArch, aby sme náš príklad mohli animovať.

Kód 3.1: Telo metódy simulateByInput()

```

1 type = string(read("Prosím, zadajte typ biomu písmenom D (
    desert), T (tundra) alebo F (forest) v dvojitéch úvodzovkách"
    ));
2 create object instance client_1 of Client;
3 client_1.getResultByInput(type);
  
```

V kóde 3.1 je uvedená metóda **simulateByInput()** z triedy **Example**. Jej cieľom je vytvoriť bióm podľa typu prečítaného zo vstupu od užívateľa. Trieda **Example** obsahuje aj ďalšie funkcie **simulateDesert()**, **simulateTundra()** a **simulateForest()**, ktoré majú rovnaký kód, s výnimkou priamo určeného typu v kóde: type = "D" pre púšť, type = "T" pre tundru a type = "F" pre les. Metóda **simulateScript()** vytvorí všetky typy biómov pomocou metódy **getResult()** v triede klienta.

Kód 3.2: Telo metódy getResultByInput(string biomeType)

```

1 create object instance biomeDirector of BiomeDirector;
2 if (biomeType == "D")
3     create object instance desertBuilder of DesertBuilder;
4     biomeDirector.builder = desertBuilder;
5 elif (biomeType == "T")
6     create object instance tundraBuilder of TundraBuilder;
7     biomeDirector.builder = tundraBuilder;
8 else
9     create object instance forestBuilder of ForestBuilder;
10    biomeDirector.builder = forestBuilder;
11 end if;
12 biomeDirector.build();

```

V metódach triedy **Example** sa podľa zvoleného biómu zavolá metóda **getResultByInput(biomeType)**, ktorej implementácia je v kóde 3.2. Podľa vstupného parametra biomeType sa zvolí správny staviteľ, ktorého si riaditeľ uloží do členskej premennej. Na riaditeľovi sa zavolá metóda build().

Kód 3.3: Telo metódy build()

```

1 create object instance biome of Biome;
2 builder.result = biome;
3 builder.setTerrain();
4 builder.addPlants();
5 builder.addAnimals();

```

Metóda **build()** v kóde 3.3 inicializuje inštanciu triedy **Biome** a pomocou vlastného staviteľa zavolá metódy na pridanie terénu, rastlín a zvierat, čím vytvorí výsledný bióm. Ten je možné neskôr získať pomocou metódy v staviteľovi: getResult().

Kód 3.4: Metódy **setTerrain()**, **addPlants()**, **addAnimals()** pre triedu DesertBuilder

```

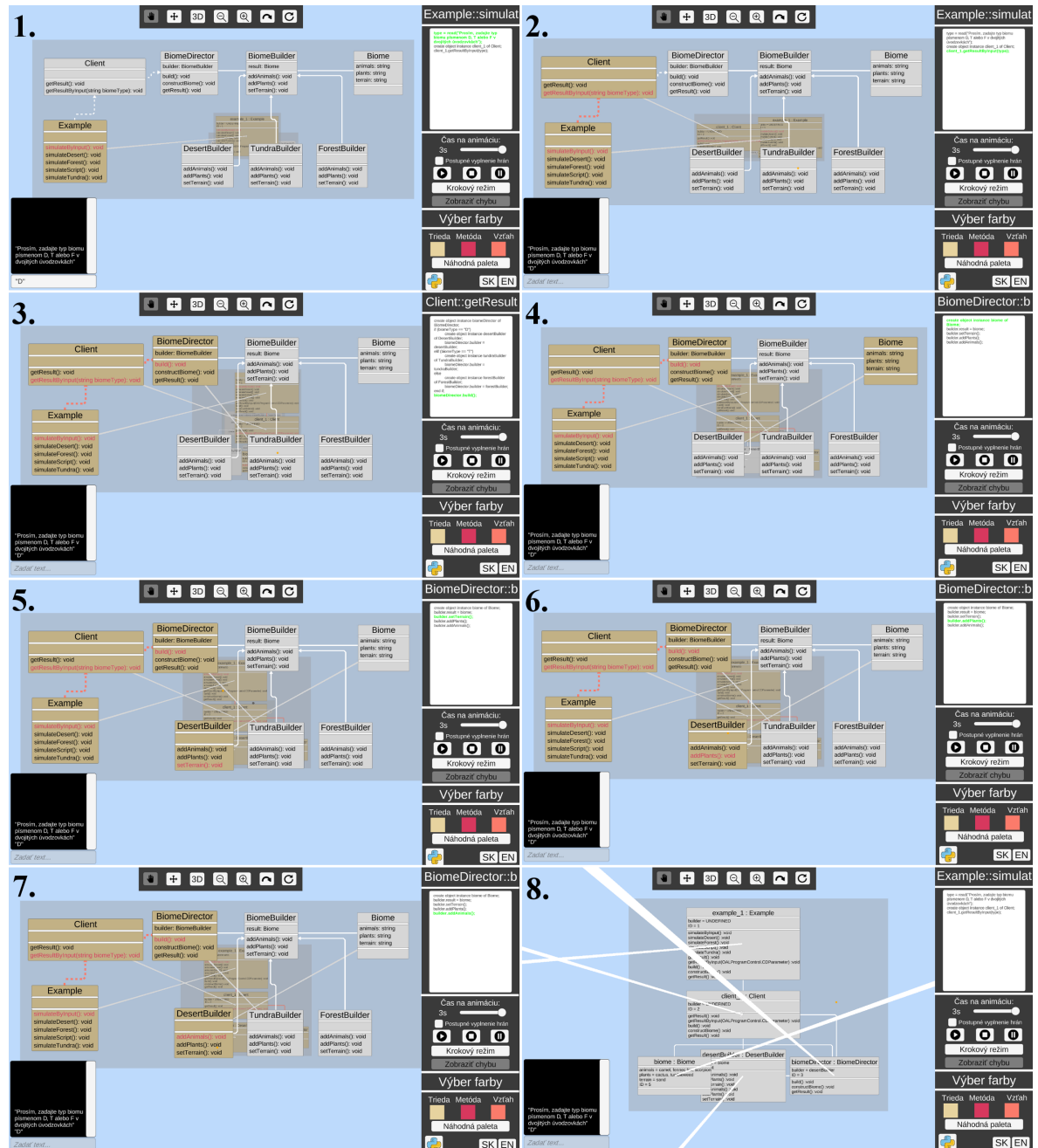
1 setTerrain():
2     result.terrain = "sand";
3
4 addPlants():
5     result.plants = "cactus, tumbleweed";
6
7 addAnimals():
8     result.animals = "camel, fennec fox, scorpion";

```

Jednotlivé implementácie metód **setTerrain()**, **addPlants()** a **addAnimals()** sú implementované rovnako vo všetkých staviteľoch, ale s odlišným obsahom. V kóde 3.4

je príklad pre púšť. Pre les a tundru by to bol iný pre nich vhodný terén, rastliny a zvieratá.

Animácia



Obr. 3.2: Animácia prvej implementácie vzoru Staviteľ v nástroji AnimArch

V triede **Example** sme pripravili počiatočné metódy pre animácie. Prvou je **simulateByInput()**, ktorá vyžaduje voľbu od užívateľa. Ostatné metódy majú bióm pevne daný, pričom metóda **simulateScript()** vytvorí jeden bióm s každým staviteľom.

V nástroji AnimArch spustíme vytvorený kód s počiatočnou metódou **Simulate-**

ByInput(). Pri animácii sa metóda vyznačí na červeno, ako je vidieť na snímke 1. na obrázku 3.2. Zároveň sa v konzole vypíše inštrukcia pre užívateľa, aby si vybral typ biómu. Na 2. snímke animácie je označená metóda **getResultByInput()** triedy **Client**, ktorá spracuje vstup od užívateľa a na triede **BiomeDirector** zavolá metódu **build()** (3. snímka). Táto metóda inicializuje výsledný produkt typu **Biome** - trieda je podfarbená na 4. snímke. Následne môže užívateľ pozorovať, ako sa postupne z riaditeľa vykonávajú metódy vo vybranom staviteľovi **DesertBuilder**. Na snímkach 5. - 7. je najskôr na červeno zvýraznená metóda **setTerrain()**, potom **setPlants()** a nakoniec **setAnimals()**. Na objektovom diagrame na 8. snímke sa nachádza výsledná inštancia triedy **Biome**, ktorý obsahuje vybranému biómu prislúchajúci obsah v premenných: "terrain = sand", "plants = cactus, tumbleweed" a "animals = camel, fennec fox, scorpion".

Diskusia

Prvá implementácia vzoru Staviteľ nezodpovedá najlepšie jeho definícii. Ako sme uviedli v časti 1.4, vzor staviteľ sa používa vtedy, keď je vytvorenie celého komplexného objektu nezávislé od tvorby jeho častí a spôsobu ich zostavenia. Bióm sme zatiaľ zostavovali rovnakým spôsobom - metódou **build()** v triede **BiomeDirector** - teda výsledok mal vždy rovnakú štruktúru. Napriek tomu navrhujeme prvú implementáciu ponechať a využiť ju ako medzikrok v procese zoznámenia sa so vzorom Staviteľ.

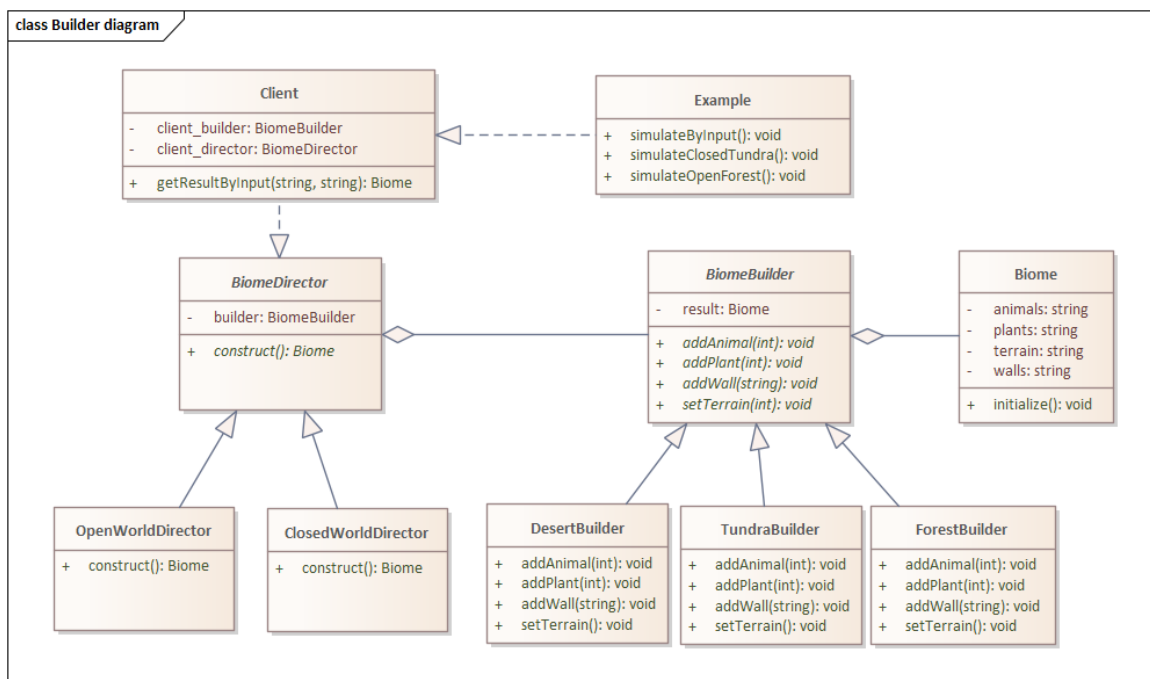
3.3 Druhá implementácia

V ďalších podkapitolách sa budeme venovať druhej implementácii vzoru Staviteľ.

Triedny diagram

Ako je vidieť na obrázku 3.3, do triedneho diagramu sme pridali triedy **OpenWorldDirector** a **ClosedWorldDirector**, ktoré dedia od pôvodnej triedy **BiomeDirector**. Ich metódy sme obmedzili na jedinú: **construct()**. Implementujú postup vytvorenia produktu typu **Biome**. Do triedy **Biome** sme pridali parameter, ktorý definuje prítomnosť stien. Aj Staviteľia majú metódu na vytvorenie stien: **addWall()**.

Podľa zvoleného riaditeľa sa vytvorí otvorený alebo uzavretý svet. Odstránili sme metódy na simuláciu jednotlivých biómov z triedy **Example**.



Obr. 3.3: Triedny diagram vzoru Staviteľ

OAL kód

Metódy z prvej implementácie upravíme a doplníme o nové metódy, aby zodpovedala novému prístupu s dvoma typmi riaditeľa.

Kód 3.5: Telo metódy simulateByInput()

```

1 biome = string(read("Prosím, zadajte typ biomu písmenom D (
    desert), T (tundra) alebo F (forest) v dvojitéch úvodzovkách"
));
2 director = string(read("Prosím, zadajte typ levelu písmenom O (
    open - survival) alebo C (closed level) v dvojitéch úvodzovkách"
));
3 create object instance client of Client;
4 client.getResultByInput(biome, director);
  
```

V ukážke kódu 3.5 je implementácia metódy **simulateByInput()** triedy **Example**. Túto metódu sme rozšírili, aby si užívateľ dokázal vybrať aj typ levelu. V prvom riadku si kód požiada o vstup, aby vybral bióm a hneď na to v druhom riadku typ levelu. Následne na objekte typu **Client** zavolá metódu **getResultByInput(biomeType, directorType)**, ktorej posunie oba tieto parametre.

Kód 3.6: Telo metódy getResultByInput(biomeType, directorType)

```

1 if (directorType == "O")
2     create object instance self.client_director of
        OpenWorldDirector;
3 else
4     create object instance self.client_director of
        ClosedWorldDirector;
5 end if;
6 if (biomeType == "D")
7     create object instance self.client_builder of DesertBuilder;
8 elif (biomeType == "T")
9     create object instance self.client_builder of TundraBuilder;
10 else
11     create object instance self.client_builder of ForestBuilder;
12 end if;
13 self.client_director.builder = self.client_builder;
14 return self.client_director.construct();

```

V ukážke kódu 3.6 je telo metódy **getResultByInput()** triedy **Client**, ktorá ako parameter dostane "directorType" a "biomeType" v tvare reťazca dĺžky 1 obsahujúci veľké písmeno. Na riadkoch 1 až 5 sa podľa parametra typu riaditeľa (angl. "director-Type") vytvorí inštancia príslušnej triedy otvoreného alebo uzavretého riaditeľa a uloží sa do členskej premennej "client_director".

Na riadkoch 6 až 12 sa podľa parametra predstavujúci bióm vytvorí inštancia jednej z podtried triedy **BiomeBuilder**, podobne ako v prvej implementácii a uloží sa do členskej premennej s názvom client_builder. Následne sa na riadku 13 priradí tento client_builder do členskej premennej builder v inštancii client_director.

Na poslednom riadku sa vyvolá metóda **construct()** na riaditeľovi uloženom v členskej premennej.

Kód 3.7: Telo metódy **construct()** triedy **OpenWorldDirector**

```

1 create object instance biome of Biome;
2 biome.initialize();
3 builder.result = biome;
4 builder.setTerrain();
5 repeat = 0;
6 while (repeat < 3)
7     index = 1;
8     while (index < 3)
9         builder.addPlant(index);
10        index = index + 1;

```



```

11     end while;
12
13     index = 1;
14     while (index < 4)
15         builder.addAnimal(index);
16         index = index + 1;
17     end while;
18 repeat = repeat + 1;
19 end while;
20 return biome;

```

V ďalšej ukážke kódu 3.7 sa nachádza telo metódy **construct()** triedy **OpenWorldDirector**. V prvých troch riadkoch vytvorí inštanciu triedy **Biome**, inicializuje jej premenné **animals**, **plants** a **terrain** na prázdne reťazce a uloží si jej referenciu do členskej premennej **result**. Na riadku 4 nastaví terén podľa zvoleného biómu. Na riadkoch 5 až 19 v troch cykloch pridáva do biómu rastliny a zvieratá pomocou volaní metód vybraného staviteľa **addPlant()** a **addAnimal()**. Na konci vráti vytvorenú inštanciu triedy **Biome**.

Kód 3.8: Telo metódy **construct()** triedy **ClosedWorldDirector**

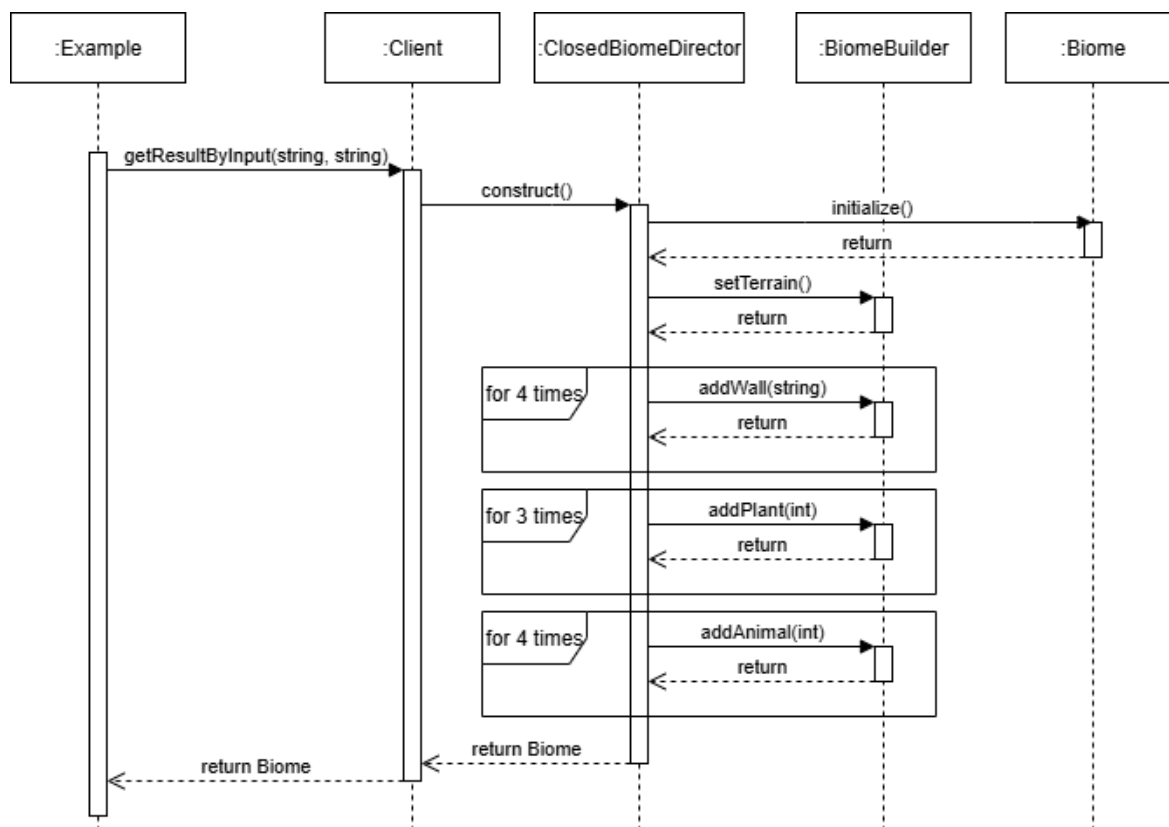
```

1 create object instance biome of Biome;
2 biome.initialize();
3 builder.result = biome;
4 builder.setTerrain();
5 builder.addWall("South");
6 builder.addWall("East");
7 builder.addWall("North");
8 builder.addWall("West");
9 index = 1;
10 while (index < 3)
11     builder.addPlant(index);
12     index = index + 1;
13 end while;
14 index = 1;
15 while (index < 4)
16     builder.addAnimal(index);
17     index = index + 1;
18 end while;
19 return biome;

```

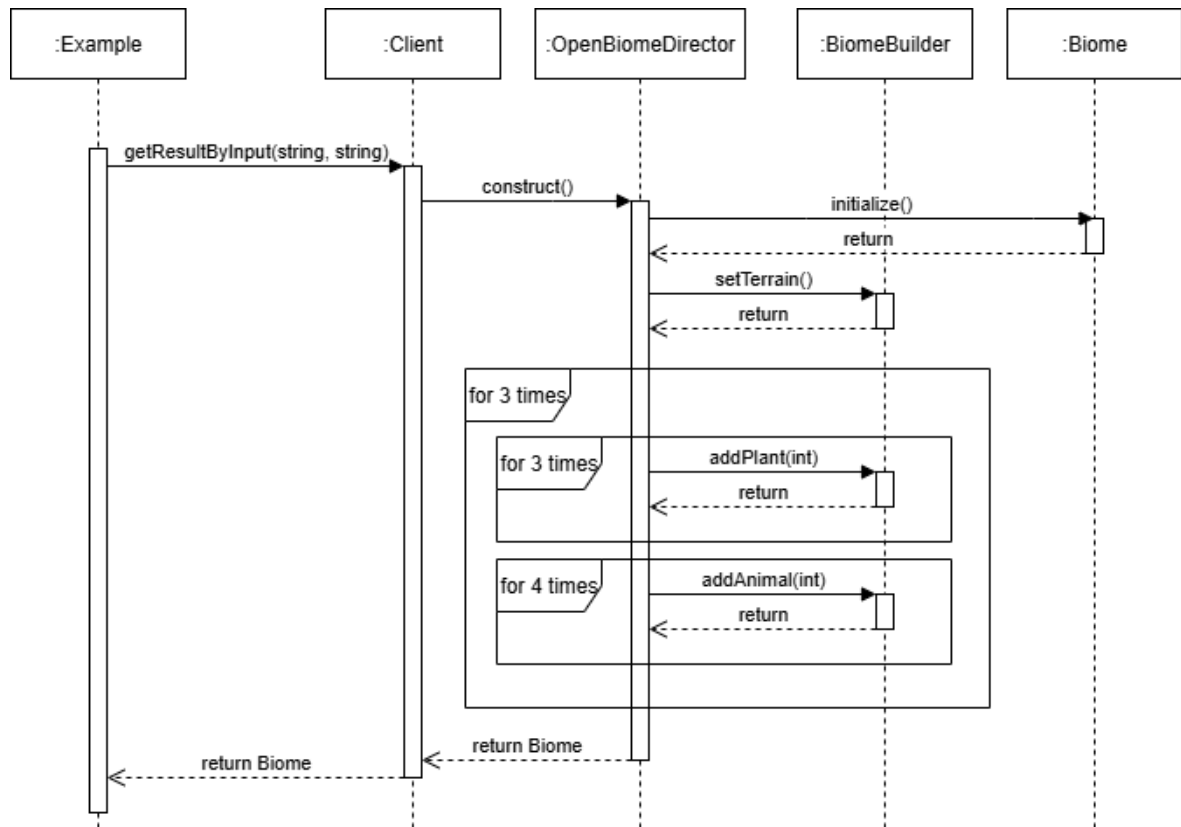
Metóda **construct()** v triede **ClosedWorldDirector** zapísaná v ukážke kódu 3.8 sa líši od predchádzajúcej metódy tým, že pridáva ohraničenia volaním metódy **ad-**

dWall(). Okrem toho pridáva rastliny a zvieratá iba v jednom cykle, čím odkazuje na to, že na ohraničenom priestore sa vygeneruje menej zvierat a rastlín ako v otvorenom svete.



Obr. 3.4: Sekvenčný diagram príkladu vzoru Staviteľ s uzavretým levelom

Porovnanie dvoch spôsobov vytvárania biómu je možné vidieť na obrázkoch 3.4 a 3.5. V týchto sekvenčných diagramoch je znázornený priebeh animácie a volaní v kóde medzi entitami. Obrázok 3.4 je s výberom uzavretého typu levelu a obrázok 3.5 s otvoreným typom sveta.



Obr. 3.5: Sekvenčný diagram príkladu vzoru Staviteľ s otvoreným levelom

Kód 3.9: Telo metódy **setTerrain()** triedy **TundraBuilder**

```

1 setTerrain()
2     result.terrain = "permafrost";

```

Kód 3.10: Telo metódy **addPlant()** triedy **TundraBuilder**

```

1 addPlant(index: integer)
2     plantsToAdd = "lichen;moss;dwarf birch";
3     plantArray = plantsToAdd.Split(";");
4
5     if (result.plants != "")
6         result.plants = result.plants + ", ";
7     end if;
8
9     result.plants = result.plants + plantArray[index - 1];

```

Kód 3.11: Telo metódy **addAnimal()** triedy **TundraBuilder**

```

1 addAnimal(index: integer)
2     animalsToAdd = "arctic fox;caribou;snowy owl";
3     animalArray = animalsToAdd.Split(";");

```

```

4
5     if (result.animals != "")
6         result.animals = result.animals + ", ";
7     end if;
8
9     result.animals = result.animals + animalArray[index - 1];

```

Kód 3.12: Telo metódy **addWall()** triedy **TundraBuilder**

```

1 addWall(direction: string)
2     if (result.walls != "")
3         result.walls = result.walls + ", ";
4     end if;
5
6     result.walls = direction + " ice block wall";

```

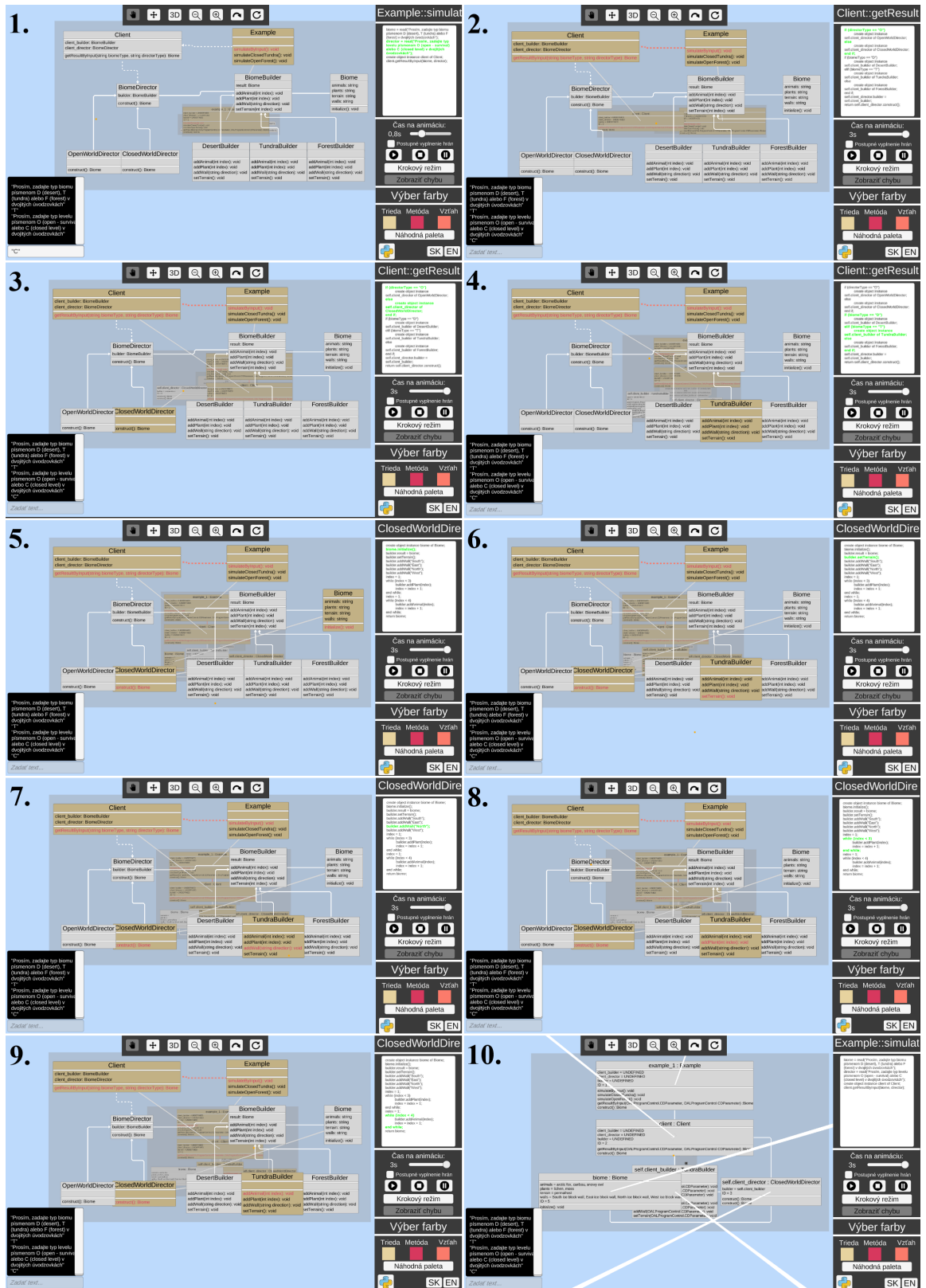
V úsekoch kódov 3.9, 3.10, 3.11 a 3.12 sú metódy triedy **TundraBuilder**. Podobne ako v prvej implementácii, pre ostatných Staviteľov ide o štruktúrovo rovnaký kód, ale s inými hodnotami, ktoré sa pridávajú do atribútov biómu prislúchajúce k danému typu biómu podľa poradia kedy sú pridané. Pripravené sú v reťazci "animalsToAdd" a "plantsToAdd". V konečnom dôsledku obsahujú atribúty výsledného produktu terén, rastliny, zvieratá a steny zvoleného typu biómu a to v počte, ktorý určil staviteľ.

Animácia

Pre animáciu Staviteľa sme vytvorili počiatočné metódy v triede **Example**. Metóda **simulateByInput()** vyžaduje v konzole vstup od užívateľa kvôli výberu typu levelu a biómu. Metódy **simulateOpenForest()** a **simulateClosedTundra()** majú tieto parametre napevno určené, čo vyplýva z ich názvu.

Na obrázku 3.6 je zobrazená animácia práve s počiatočnou metódou **simulateByInput()** v nástroji AnimArch. Na snímke 1. si užívateľ zvolil bióm tundra a uzavretý typ sveta a začalo sa vykonávanie (snímka 2.). Bola vytvorená inštancia triedy **ClosedWorldDirector** (3. snímka) a triedy **TundraBuilder** (4. snímka). Po spustení metódy **build()** v riaditeľovi sa vytvorila inštancia triedy **Biome** a jej členské premenné sa inicializovali na prázdne reťazce v metóde **initialize()** na 5. snímke. Následne sa nastavil terén - vyznačená je metóda **setTerrain()** v staviteľovi (6. snímka). Vpravo hore na snímkach 6 až 9 je aj samotný kód procesu stavania. Ako sa vytvárali 4 steny, zakaždým zablikala metóda **addWall()** v staviteľovi (snímka 7). Rovnako podľa určeného počtu cyklov metódy **addPlants()** a **addAnimals()** (snímky 8. a 9.). Na 10. snímke sa nachádza objektový diagram, na ktorom vidíme výslednú inštanciu triedy **Biome**. Tá obsahuje hodnoty premenných zodpovedajúce zvolenému biómu tundry v

uzavretom leveli: "animals = arctic fox, caribou, snowy owl", "plants = lichen, moss", "terrain = permafrost" a "walls = South ice block wall, East ice block wall, North ice block wall, West ice block wall".



Obr. 3.6: Animácia druhej implementácie vzoru Staviteľ v nástroji AnimArch

Diskusia

Druhá implementácia vzoru staviteľ zodpovedá návrhovému vzoru Staviteľ. V našom príklade je možné vytvoriť 6 kombinácií sveta z troch biómov a dvoch typov levela. Výsledkom je inštancia typu **Biome**, na ktorej môže užívateľ vidieť výsledok na základe ním zvolenej kombinácie pred začiatkom animácie. Užívateľ si môže vyskúšať všetkých 6 kombinácií.

Kapitola 4

Vzor Most

V tejto kapitole sa venujeme návrhu a implementácii príkladu pre návrhový vzor Bridge (opísaný v podkapitole 1.4).

4.1 Návrh

Pre vzor Most navrhujeme príklad, ktorý využije tematiku objednávanie a doručovania jedla v reštaurácii. Abstrakciu reprezentuje spôsob objednania a implementáciu spôsob doručenia.

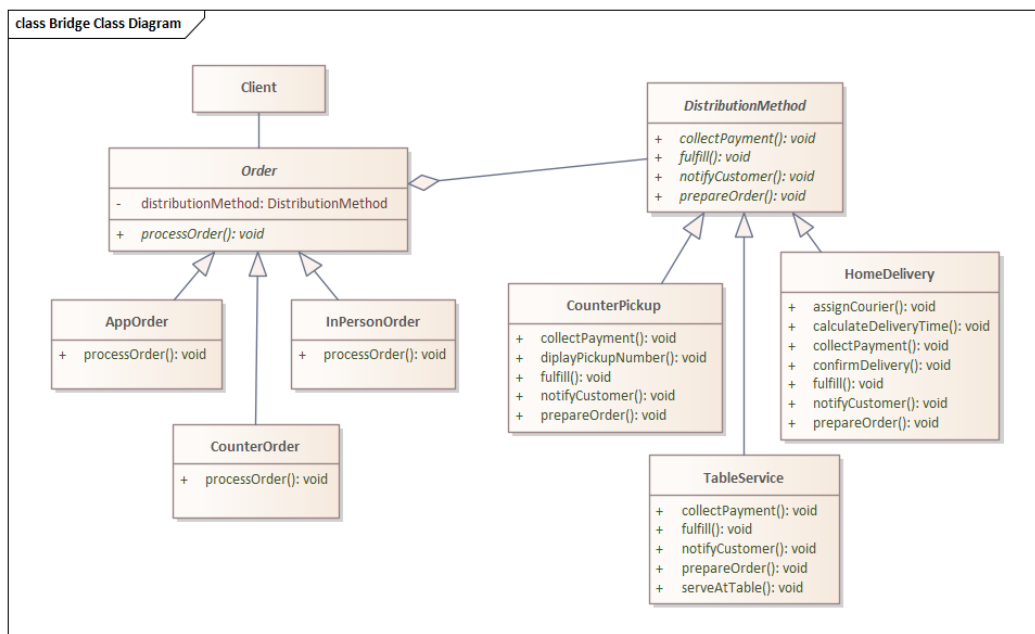
Objednávanie jedla je bežnou súčasťou v dnešnom svete, náš príklad teda prináša...

4.2 Implementácia

V nasledujúcich kapitolách sa venujeme implementácii nášho príkladu pre vzor Most.

Triedny diagram

Triedny diagram pozostáva z dvoch hierarchií tried. Abstraktná trieda **Order** predstavuje objednávku a abstraktná trieda **DistributionMethod** spôsob realizácie objednávky. Trieda **Order** má odkaz na príslúchajúcu inštanciu triedy **DistributionMethod**, práve ako to má vo vzore Most byť. Dedia od nej triedy **AppOrder**, **InPersonOrder** a **CounteOrder**, ktoré v našom príklade reprezentujú spôsob v tomto poradí objednanie jedla cez aplikáciu, osobne u čašníka a pri pulte. Na druhej strane od triedy **DistributionMethod** dedia triedy **CounterPickup**, **HomeDelivery** a **TableService**. Tie predstavujú v rovnakom poradí vyzdvihnutie pri pulte, doručenie domov a prinesenie na stôl.



Obr. 4.1: Triedny diagram vzoru Most

OAL kód

Animácia

Kapitola 5

Štýl Potrubia a filtre

V tejto kapitole sa venujeme návrhu a implementácii príkladu pre architektonický štýl Potrubia a filtre (opísaný v podkapitole 1.5).

5.1 Návrh

Ako príklad štýlu Potrubia a filtre navrhujeme použiť v operačnom systéme typu Unix bežne využívanú Rúru. Rúra je prepojenie viacerých príkazov tak, že vstupom do nasledujúceho príkazu je výstup predchádzajúceho príkazu. Užívateľ by mal byť schopný zadať príkaz autenticky ako v konzole operačného systému Unix. Navrhujeme vybrať malú podmnožinu príkazov s prepínačmi, ktoré budú postačujúce na simuláciu základných príkladov a zároveň dostatočne pestré pre porozumenie štýlu.

Rúra v Linuxe je nástroj, ktorý študenti informatických odborov väčšinou poznajú, či už z predchádzajúceho vzdelávania, alebo z praxe. Preto by im mal byť náš príklad blízky a známy.

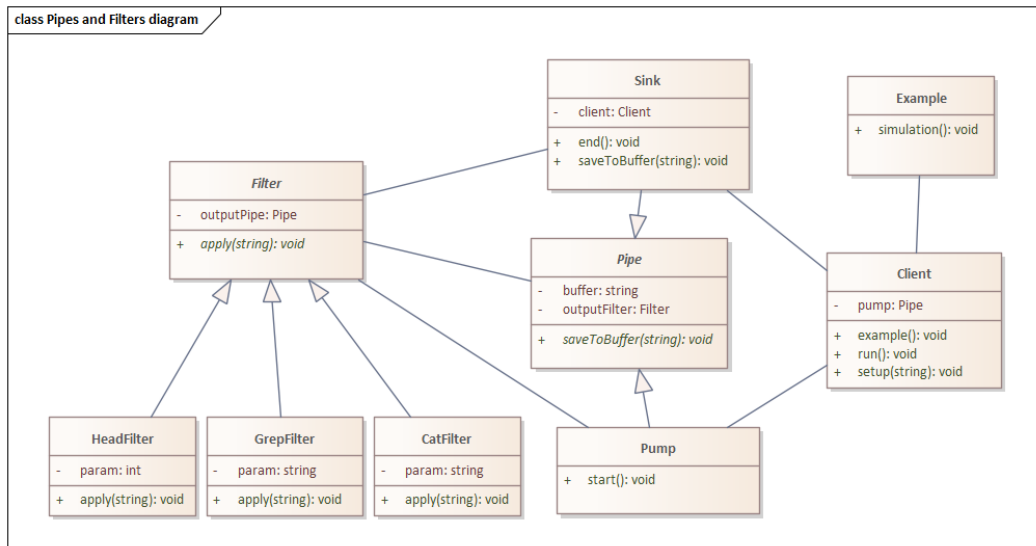
5.2 Prvá implementácia

V podkapitolách sa budeme venovať prvej implementácii štýlu Potrubia a Filtre s tromi typmi filtrov a pokusom o tzv. "push" pri vykonávaní.

Triedny diagram

V prvom návrhu triedneho diagramu na obrázku 5.1 sme vyjadrili vzťahy medzi komponentmi potrebnými pre náš príklad. Osobitnú triedu sme definovali pre Potrubie a pre Filter. Od filtra dedia tri triedy: **HeadFilter**, **GrepFilter** a **CatFilter**, ktoré obsahujú konkrétne implementácie úpravy vstupu. **HeadFilter** má ako parameter číslo a teda posúva po potrubí daný počet riadkov zo vstupu. **GrepFilter** vyberá len riadky,

ktorý obsahujú podreťazec zadefinovaný ako parameter filtra. **CatFilter** zapisuje výsledok do súboru. Triedy **Pump** a **Sink** reprezentujú v uvedenom poradí vstupné a výstupné potrubie a teda dedia od triedy **Pipe**. Trieda klienta sa stará o čítanie zo vstupu, prípravu štruktúry potrubí a filtrov a spúšťanie simulácie. Trieda **Example** obsahuje predpripravené vstupy a volá metódy triedy **Client**.



Obr. 5.1: Prvý návrh triedneho diagramu štýlu Potrubia a Filtre

OAL kód

V triede **Example** simulujeme príklad, ktorý používa všetky tri filtre. Metóda **simulation()** v kóde 5.1 vytvára inštanciu triedy **Client** a vyvolá na ňom metódu **setup(input)** s napevno daným vstupom, ktorá ho spracuje a metódu **run()**, ktorá spustí vykonávanie.

Kód 5.1: Telo metódy **simulation()** triedy **Example**

```
1 create object instance client of Client;
2 input = "grep 'a' | head -6 | cat > 'output.txt'";
3 client.setup(input);
4 client.run();
```

V metóde **setup()** triedy **Client** zapísanej v pseudokóde 5.2 najskôr spracujeme vstup, aby sme vedeli jednoznačne určiť príkazy a ich parametre. Pre príkazy "grep" a "cat" sú parametre v jednoduchých úvodzovkách, ale pre "head" je číslo za znakom pomlčky. Navyše pre "cat" sa nachádza parameter až za znakom väčší >. Na konci cyklu máme teda pole "commands" v ktorom sú reťazce s príkazom a parametrom oddelené znakom §.

Následne v cykle prechádzame jednotlivé príkazy a podľa mena príkazu vytvoríme prislúchajúcu inštanciu filtra, teda **HeadFilter**, **CatFilter** alebo **GrepFilter** a na-

stavíme im prislúchajúci parameter do členskej premennej "param". Zároveň medzi filtrami vytvárame potrubia - inštancie triedy **Pipe**. Priebežne všetky prepájame tak, aby každé potrubie poznalo nasledujúci filter a každý filter nasledujúce potrubie.

Kód 5.2: Telo metódy **setup()** triedy **Client**

```
1 commands => parse input into list
2 create object pump of Pump
3 help => pump
4
5 for each (cmd, param) in commands do
6     filter <= create corresponding Filter with param
7     help.outputFilter <= filter
8
9     if cmd is not last then
10         pipe <= create Pipe
11         filter.outputPipe <= pipe
12         help <= pipe
13     else
14         sink <= create Sink
15         filter.outputPipe <= sink
16         sink.client <= self
17     end if
18 end for
```

V kóde 5.3 sú telá metód, ktoré zabezpečujú vykonávanie príkazov, ktoré by malo reprezentovať spôsob nazývaný "push". Vykonávanie začína vyvolaním metódy **start()** na prvom potrubí, konkrétne inštancie triedy **Pump**. Tá spustí vykonávanie nasledujúceho filtra, teda metódu **apply(buffer)** s obsahom svojho zásobníka ako parametra. Pre každý filter platí, že keď ukončí svoje vykonávanie, uloží výsledok do zásobníka nasledujúceho potrubia, ktoré opäť spustí vykonávanie nasledujúceho filtra. Tak to pokračuje až do momentu keď po filtri nasleduje potrubie triedy **Sink**, ktorý uloží dáta do zásobníka a vypíše výsledok do konzoly.

Kód 5.3: Telá metód, ktoré zabezpečujú vykonávanie príkazov

```
1 Client.run():
2     self.pump.start();
3
4 Pump.start():
5     self.outputFilter.apply(self.buffer);
6
7 Pipe.saveToBuffer(data):
8     self.buffer = data;
```

```

9      self.outputFilter.apply(self.buffer);
10
11 Sink.saveToBuffer(data):
12     self.buffer = data;
13     self.end();
14
15 Sink.end():
16     write(self.buffer);

```

Implementácia metódy **apply()** samotných filtrov ja popísaná v pseudokóde 5.4. Pre **HeadFilter** si rozdelíme vstup na riadky a kým nedosiahneme počet v atribúte "param" alebo posledný riadok, tak daný riadok pridávame do výstupu. Ten nakoniec spojíme do jedného reťazca. Filter teda posunie ďalej konkrétny počet riadkov od začiatku vstupu.

GrepFilter má rovnakú štruktúru, ale do výstupu zahrnie len riadky, ktoré obsahujú reťazec v atribúte "param". Pre triedu **CatFilter** prebehne zápis obsahu do súboru v priečinku predpripravenom na tento účel a to na ceste "Assets\Resources\Files\". Ide o relatívnu cestu vzhľadom na priečinok, v ktorom je uložený projekt.

Na konci každého filtra je zavolaná metóda **saveToBuffer()**, ktorá nie len uloží výstup do zásobníka nasledujúceho potrubia, ale aj zabezpečí ďalšie vykonávanie.

Kód 5.4: Telá metód **apply()** pre filtre tried **HeadFilter**, **CatFilter** a **GrepFilter**

```

1 HeadFilter.apply(input):
2     lines <= split input by newline
3     result <= first param lines from lines
4     outputPipe.saveToBuffer(join result by newline)
5
6 GrepFilter.apply(input)
7     lines <= split input by newline
8     result <= all lines containing param
9     outputPipe.saveToBuffer(join result by newline)
10
11 CatFilter.apply(input)
12     write input to file "Assets/Resources/Files/" + param
13     outputPipe.saveToBuffer(input)

```

Animácia

Diskusia

V animácii v AnimArch-u je vidieť, že sa postupne zvyrazňujú potrubia a filtre a až po ukončení vykonávania postupne zhasnú. To môže užívateľa pomýliť v tom, ako by mal tento štýl naozaj fungovať. Podobne ako snaha o tzv. "push" by vyzeral spôsob "pull", preto sa ho nebudeme pokúšať implementovať. Ak by sme to ale chceli implementovať viac realisticky, jednotlivé filtre by mali v samostatných vetvách kontrolovať, či majú nejaký vstup a teda, či môžu spustiť svoju metódu **apply()**. Tento prístup by spôsobil náhodné zvyrazňovanie až blikanie filtrov v animácii, čo by mohlo opäť užívateľa zmiestiť. Navrhujeme trochu priamočiarejší prístup.

5.3 Druhá implementácia

Na základe diskusie k prvej implementácii navrhujeme upraviť implementáciu tak, že pridáme triedu **Manager**, ktorá bude podľa očakávania riadiť aktiváciu potrubí a filtrov. Veríme, že tak dostaneme viac plynulú a priamočiaru animáciu vykonávania. Detaily úprav opisujeme v nasledujúcich podkapitolách.

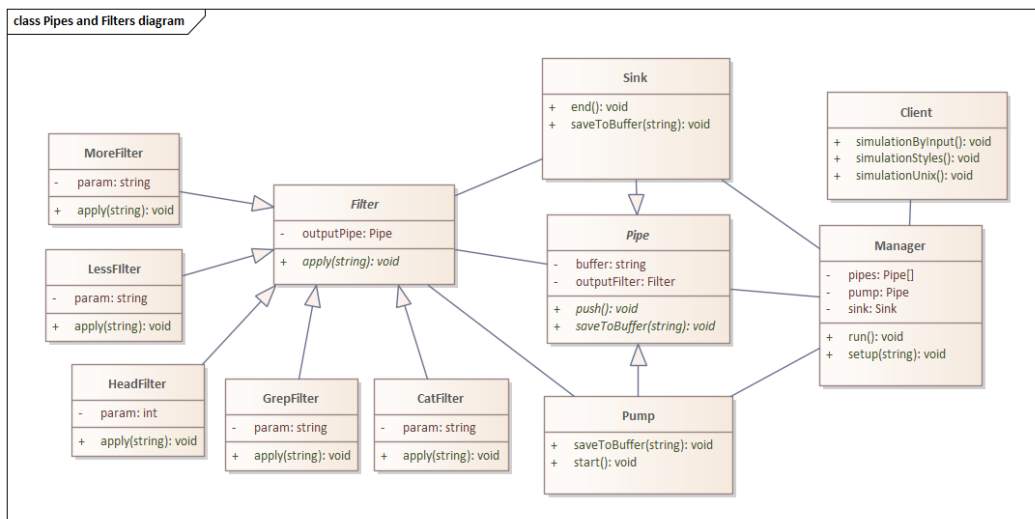
Triedny diagram

Aby animácia pôsobila viac ako postupné aplikovanie filtrov, pridali sme manažéra (trieda **Manager**), ktorý podľa štruktúry potrubí a filtrov vyvoláva ich vykonávanie. Okrem toho sme pridali dva nové filtre: **LessFilter** a **MoreFilter** pre širšie možnosti príkladov. **LessFilter** prečíta priamo obsah vstupu a posunie ho po potrubí. **MoreFilter** prečíta obsah súboru, ktorého názov dostane ako parameter. Celú animáciu je možné spúšťať z triedy **Client**. Viď obrázok s diagramom 5.2.

OAL kód

Hlavná zmena oproti prvej implementácii bolo zapojenie triedy **Manager**, ktorá prevzala metódy **setup()** a **run()**. Metóda **setup** sa v zásade ne zmenila, iba sme pridali každé nové potrubie do členskej premennej typu pole triedy **Manager**. V metóde **run** teda môžeme prechádzať týmto poľom a postupne spúšťať vykonávanie filtrov ako je vidieť v 5.5. Na spustenie sa používa metóda **push()**, ktorá spôsobí aplikáciu filtra na vstup. Pre filter stále platí, že na konci uloží výstup do nasledujúceho potrubia. Tu sa však vykonávanie konkrétneho filtra ukončí a ďalší postup opäť spravuje trieda **Manager** metódou **run()**.

Kód 5.5: Telá metód, ktoré zabezpečujú vykonávanie príkazov



Obr. 5.2: Druhý návrh triedneho diagramu štýlu Potrubia a Filtre

```

1 Manager.run():
2     for each pipe in self.pipes
3         pipe.push();
4     end for;
5     self.sink.end();
6
7 Pipe.push():
8     self.outputFilter.apply(self.buffer);
9
10 Pipe.saveToBuffer(data):
11     self.buffer = data;
12
13 Sink.end():
14     write(self.buffer);
  
```

Okrem toho sme pridali dva filtre: **MoreFilter** a **LessFilter**. Telá ich metódy **apply()** uvádzame v 5.6. **LessFilter** jednoducho prevezme vstup z pôvodného príkazu a posunie ho ďalej. **MoreFilter** prečíta obsah súboru.

Kód 5.6: Telá metód apply pre triedy **LessFilter** a **MoreFilter**

```

1 LessFilter: apply(input):
2     self.outputPipe.saveToBuffer(input);
3
4 MoreFilter: apply(input):
5     path = "Assets\Resources\Files\" + param;
6     content = ReadFile(path);
7     self.outputPipe.saveToBuffer(content);
  
```

Pre pohodlnejšie spúšťanie animácie sme pripravili 3 simulácie v triede **Client** v 5.7. Metóda **simulationStyles()** používa vstup so zoznamom architektonických štýlov, vyberie taký, ktorý obsahuje reťazec "Pipe" a prvý z nich uloží do súboru "output.txt". Metóda **simulationUnix()** sa zameriava na reálne použitie, kde sa ako vstup zoberú procesy a vyberú prvé dva, ktoré patria užívateľovi "user". Nakoniec pre väčšiu flexibilitu ostáva metóda **simulationByInput()**, ktorej môže užívateľ zadať svoj vlastný príkaz typu Unix rúry s použitím našich piatich existujúcich filtrov.

Kód 5.7: Telá metód simulácií triedy **Client**

```
1 Client.simulationStyles():
2     input = "more 'input.txt' | grep 'Pipe' | head -1 | cat
3         > 'output.txt'";
4     // nastavenie a spustenie animácie
5 Client.simulationUnix():
6     input = "more 'processes.txt' | grep 'user' | head -2 |
7         cat > 'user_processes.txt'";
8     // nastavenie a spustenie animácie
9 Client.simulationByInput():
10    input = string(read("Prosím, zadajte Unix piping príkaz (
11        more, less, grep, head, cat) v dvojitéch úvodzovkách.
12        Napríklad: \"more 'processes.txt' | grep 'user' | head
13        -2 | cat > 'user_processes.txt'\""));
14    // nastavenie a spustenie animácie
```

Animácia

Zvýrazňujú sa po trojiciach

Diskusia

Druhá implementácia vizuálne lepšie predstavuje štýl Potrubia a Filtre, Postupné spúšťanie filtrov a posúvanie upraveného vstupu cez zásobníky jasne ukazuje ako sa tento text v čase modifikuje. Okrem toho je pripravená na prácu s piatimi filrami, čo rozširuje možnosti animácií. Navyše kód obsahuje už pripravené príklady.

Kapitola 6

Evaluácia

Kapitola 7

Manuál

Záver

Literatúra

- [1] Gme: Generic modeling environment.
- [2] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley Object Technology Series. Addison-Wesley, Reading, Massachusetts, first edition, 1998. 6th printing, April 2000.
- [3] África Domingo, Jorge Echeverría, Oscar Pastor, and Carlos Cetina. Evaluating the benefits of model-driven development: empirical evaluation paper. In *International Conference on Advanced Information Systems Engineering*, pages 353–367. Springer, 2020.
- [4] Matej Ferenc, Ivan Polasek, and Juraj Vincur. Collaborative modeling and visualization of software systems using multidimensional uml. In *2017 IEEE Working Conference on Software Visualization (VISSOFT)*, pages 99–103. IEEE, 2017.
- [5] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Addison-Wesley, Indianapolis, IN, 1995. 37th Printing March 2009.
- [6] Brent Hailpern and Peri Tarr. Model-driven development: The good, the bad, and the ugly. *IBM systems journal*, 45(3):451–461, 2006.
- [7] Joshua Kerievsky. *Refactoring to Patterns*. Addison-Wesley, Boston, MA, 2004.
- [8] Akos Ledeczi, Miklos Maroti, Arpad Bakay, Gabor Karsai, Jason Garrett, Charles Thomason, Greg Nordstrom, Jonathan Sprinkle, and Peter Volgyesi. The generic modeling environment. In *Workshop on Intelligent Signal Processing, Budapest, Hungary*, volume 17, page 2001, 2001.
- [9] Miguel Pinto Luz and Alberto Rodrigues Da Silva. Executing uml models. In *3rd Workshop in Software Model Engineering (WiSME 2004)*, volume 192, 2004.
- [10] OOA Tool. Oal documentation, 2008. Accessed: 21 August 2025.

- [11] Lukas Radosky and Ivan Polasek. Executable multi-layered software models. In *Proceedings of the 1st International Workshop on Designing Software*, pages 46–51, 2024.
- [12] Reinhard Schauer and Rudolf K Keller. Pattern visualization for software comprehension. In *Proceedings. 6th International Workshop on Program Comprehension. IWPC'98 (Cat. No. 98TB100242)*, pages 4–12. IEEE, 1998.
- [13] Anubha Sharma, Manoj Kumar, and Sonali Agarwal. A complete survey on software architectural styles and patterns. *Procedia Computer Science*, 70:16–28, 2015.
- [14] Sparx Systems. Sparx systems – modeling tools for business, software, and systems. <https://sparxsystems.com>, 2025. Accessed: 2025-07-04.
- [15] Nenad Ukić, Pál L Pályi, Marijan Zemljić, Domonkos Asztalos, and Ivan Markota. Evaluation of bridgepoint model-driven development tool in distributed environment. In *Workshop on Information and Communication Technologies conjoint with 19th International Conference on Software, Telecommunications and Computer Networks, SoftCOM 2011*, pages 26–30. Split: Fakultet elektrotehnike, strojarstva i brodogradnje Sveučilišta u Splitu, 2011.